

JEPA Policy: Diffusion-Free Imitation Learning via Paired Action and Future Representation Prediction

Abstract—Standard behavior cloning supervises actions without explicitly constraining the future representation paired with each demonstrated action chunk. We introduce JEPA Policy, a diffusion-free framework that uses the action chunk and its observed future representation as paired training targets. Action and future-representation tokens interact in a shared Transformer and are refined through two forward passes. Future prediction can therefore shape the representation used to generate actions. Dual-branch and gradient-routing controls attribute the gain to this shared topology rather than to an auxiliary prediction head alone. Across nine simulated tasks, JEPA Policy improves mean success over the action-only MIP baseline and outperforms Diffusion Policy under the evaluated configurations, while adding 0.29 ms to MIP’s model latency. A five-task, 630-episode physical-robot study produces the same pooled ranking. Further audits find no complete representation collapse under action supervision and identify a task-conditioned failure-ranking signal in future-prediction error. These results support paired future-representation supervision as a practical approach to low-latency visuomotor imitation without iterative generative sampling.

Index Terms—Imitation learning, joint-embedding predictive architecture (JEPA), predictive representation learning, robotic manipulation.

Anonymous review materials:

Project: anonymous.4open.science/w/JEPAPolicyPage2701/

Code: anonymous.4open.science/r/JEPAPolicy2701/

The complete source code and configurations will be released publicly upon formal publication.

I. INTRODUCTION

ROBOT imitation learning aims to learn policies capable of completing complex manipulation tasks from expert demonstrations. Advances in visual encoders, Transformer architectures, and generative action modeling have improved imitation learning on grasping, pushing, insertion, drawer opening, and long-horizon tabletop manipulation. Typical behavior cloning methods, the paradigm since ALVINN [1], formulate policy learning as a supervised problem of predicting the expert action sequence $\mathbf{a}_{t:t+H}$ directly from the current observation o_t . This paradigm is simple and efficient, but its supervision focuses on action labels and does not constrain the resulting future state. It also inherits the compounding error caused by the distribution shift of open-loop supervised imitation [2].

In robotic manipulation, however, expert actions are not isolated labels. An action sequence is useful because it advances the current state toward a task-relevant outcome. Examples include pushing an object to a target, approaching a stable grasp, or establishing tool contact. An expert demonstration segment therefore supplies two aligned labels: the expert action and the future state observed after that action. Standard behavior cloning typically uses only the action label.

Using these future observations for supervision, however, raises a modeling question. The same scene may precede a

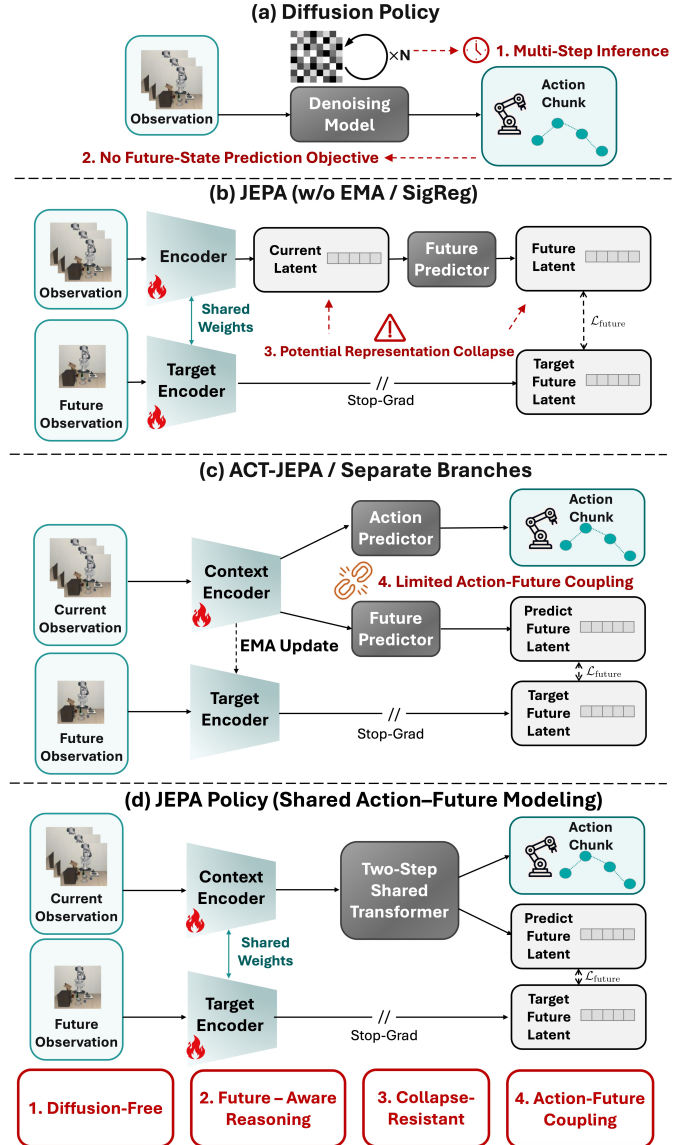


Fig. 1. Schematic comparison of design paradigms (architecture only, not a performance comparison). Red numbers mark a limitation of each prior design; the four boxes below show how JEPA Policy addresses them. Flames mark encoders trained by gradient descent. All encoders are trained from scratch because this study uses no pretrained or frozen visual backbone. “//” marks the stop-gradient.

leftward push, a rightward push, or a grasp, each leading to a different outcome. Regressing these futures from the observation alone can average incompatible targets. Such ambiguity makes explicit modeling of multiple future modes a natural consideration. Yet predicting an action’s task-relevant consequence is a different objective from generating diverse, visually detailed future videos.

Diffusion Policy [3] learns an action distribution through iterative denoising, but repeated network calls add inference latency. The study introducing the *minimal iterative policy* (MIP) [4] questions whether distribution modeling explains the success of generative control. On the evaluated behavior-cloning benchmarks, modeling multimodal action distributions did not explain the advantage of flow-based policies. Instead, supervised iterative computation and suitable training noise were the key ingredients. Its deterministic two-step MIP matched flow-based policies without fitting the full action distribution. These findings motivate a further question: can joint prediction of actions and paired future representations improve control without explicitly modeling the multimodal distribution of either output?

Our setting offers two reasons to explore this possibility further. First, each demonstration pairs an action sequence with the future observation recorded along the same trajectory. This pairing provides supervision for learning which outcome accompanies which action, rather than treating the two targets as unrelated. Second, predicting a future representation rather than a video avoids the requirement to reconstruct low-level visual detail. We therefore extend MIP's coarse-to-refined process to jointly predict actions and their paired future representations. The aim is to improve control through paired future supervision while retaining two-pass, diffusion-free inference for both outputs.

We propose **JEPA Policy**, a diffusion-free framework for joint action–future observation representation prediction (Fig. 1). Fig. 1 places the design against three reference points. Diffusion Policy [3] (a) denoises iteratively, so one decision costs N network calls. Its objective does not constrain the future reached by the predicted action. A plain latent-future JEPA [5] (b) has no action supervision or standard anti-collapse mechanism, such as an EMA target encoder or variance–covariance regularization. Its representation can therefore collapse. ACT-JEPA [6] (c) shares a context encoder but uses separate predictor and decoder branches behind an EMA target encoder. The action and future paths consequently remain loosely coupled. JEPA Policy (d) places action and future representation tokens in one shared two-step Transformer, where they interact in every layer. The same encoder produces the stopped future target, and the model uses no EMA or frozen target encoder.

Given the current observation, the model simultaneously predicts the expert action sequence and its paired future observation representation within one shared Transformer. Action and future-representation tokens share one self-attention stack and remain mutually visible in every layer, allowing the future objective to shape the action-generating representation directly during training.

Both targets are available in the original demonstrations, so the additional supervision requires no new annotation. Joint prediction encourages the policy to retain information about object position, contact, task progress, and action consequences. Unlike reconstructing a future image, matching its representation does not require reproducing texture, illumination, or background appearance.

The future prediction branch of JEPA Policy also provides

a deployment-time interface. At inference, the policy outputs the action sequence through the two-step predictor and simultaneously produces a prediction of the future observation representation. After the action is executed, the actual future observation can be mapped into representation space by the same encoder and compared with the previously predicted future representation. This future-consistency error serves as a rollout-level diagnostic signal for analyzing whether the policy is drifting away from the expert demonstration manifold, entering high-risk states, or requiring human intervention. We treat this signal as a behavior-consistency diagnostic, not as a complete uncertainty estimate.

This setting differs from purely self-supervised JEPA-style latent prediction. In pure self-supervised representation prediction, without additional anti-collapse mechanisms the model may map different inputs to similar or identical representations, forming a trivial solution. Many self-supervised JEPA or Siamese representation learning methods therefore require an EMA target encoder, a frozen target encoder, architectural asymmetry, or explicit variance–covariance regularization. In contrast, the future observation representation prediction of JEPA Policy is always optimized jointly with expert action prediction. If the shared representation collapses, object positions, contact states, and control requirements under different observations become indistinguishable, directly increasing the action prediction error. Action supervision thus provides a task-driven constraint against collapse and encourages the model to retain information needed for control. This design requires no EMA target encoder, frozen encoder, or explicit variance–covariance regularizer. Section IV-I evaluates the resulting representations. The visual encoder is initialized randomly and trained end to end with the policy rather than imported from a pretrained representation model. The observed effect therefore cannot be attributed to a frozen visual backbone that already encodes future-predictive structure. This setting also makes representation stability an optimization question for the proposed objective itself, rather than a property inherited from external pretraining.

The contributions of this paper are fourfold:

- We propose **JEPA Policy**, a future-aware robot imitation learning framework based on paired action–future observation representation supervision. Each demonstration jointly supervises an expert action sequence and its observed future representation, preserving their correspondence within the same trajectory. Architecturally, both token types occupy one shared Transformer and remain mutually visible in every self-attention layer, so the future objective directly shapes the action-generating representation.
- We extend **MIP-style coarse-to-refined prediction** to the joint generation of actions and future observation representations. A two-step diffusion-free predictor refines the action sequence and its paired future representation within one shared network. The resulting policy adds future supervision to MIP's action-only formulation without introducing a distribution-fitting objective or a denoising chain. Training and deployment remain diffusion-free, and the policy provides a future latent prediction alongside each action chunk.

- We use expert action supervision to provide a **task-driven anti-collapse constraint** for latent-space future observation representation prediction. Unlike purely self-supervised JEPA objectives, the shared representation of JEPA Policy must simultaneously support action prediction and future representation prediction; once the representation collapses, the action error immediately increases. Our visual encoder is trained end to end from scratch. The method needs neither an EMA target encoder nor a separately frozen encoder, and it uses no explicit variance–covariance regularizer.
- We propose a **deployment-time future-consistency diagnostic**. The future observation representation predicted by the policy can be compared with the actual future observation representation after execution, forming a rollout-level future-consistency error usable for failure analysis, risk monitoring, or potential human intervention.

II. RELATED WORK

A. Imitation Learning for Robotic Manipulation

Behavior cloning trains policies to match expert actions via supervised learning. Modern visuomotor imitation learning methods typically combine image encoders, Transformers, and action chunking to model temporally continuous action sequences [7], [8]. These methods require only one direct forward pass at inference. However, their training objectives focus on action matching and do not explicitly require the policy representation to encode the future outcomes produced by the demonstrated actions.

B. Diffusion Policy and Generative Action Modeling

Diffusion Policy models action generation as an iterative denoising process conditioned on observations [3]. This approach can represent complex action distributions and performs well across multiple visuomotor control tasks. Recent work attributes some gains of diffusion and flow-matching policies to noise injection and denoising training, rather than only to multimodal modeling [4]. The primary cost is that inference requires multiple network calls. Diffusion was not the first approach to multimodal action distributions. Implicit behavior cloning fits an energy-based policy [9], while Behavior Transformers discretize the action space and predict a residual offset [10]. Our additional supervision serves a different purpose. It links each demonstrated action to the future that followed it instead of modeling the distribution of possible actions.

Few- and one-step alternatives already reduce sampling cost through consistency or flow reformulations [11]–[14], while DP3 [15] reduces observation cost. We do not benchmark against them and make no relative speed claim. Our predictor is not a new accelerator; it adds paired future supervision within an existing two-pass inference budget.

C. Predictive Representation Learning and JEPA

Joint-embedding predictive architectures learn by predicting the latent representation of a target view or future observation rather than reconstructing pixels [5], [16]. This avoids

fully modeling low-level image details but requires handling representation collapse. Existing methods commonly use architectural asymmetry, stop-gradient, momentum encoders, or variance–covariance regularization [17]–[19]. Our setting differs: the predictive representation is embedded inside an imitation learning policy, and the shared representation must support accurate expert action prediction. Most related to this paper is ACT-JEPA [6], which also brings JEPA into imitation learning by jointly predicting action sequences and latent representations of future observations. Its architecture, however, routes a shared context encoder to separate observation-prediction and action-decoding branches. These branches do not interact within one attention stack, and the method relies on an EMA target encoder to prevent collapse. JEPA Policy instead uses one mutually visible stack, a stopped target as in SimSiam [20], and action supervision rather than an EMA or frozen target encoder. Because non-contrastive objectives can fail through partial rank collapse [21], [22], Sec. IV-I audits the representation spectrum. GR-1 [23] and Seer [24] also predict future visuals and actions in one Transformer. JEPA Policy instead uses a paired latent target, no video pretraining, two fixed feed-forward steps, and a shared-stack topology tested in Sec. IV-D. This distinction separates the contribution from transferring a pretrained world representation into a policy. Our experiments ask whether paired future supervision can improve action learning when the encoder and policy are trained together from scratch on the task demonstrations. They do not test the additional benefit that large-scale video or vision-language pretraining might provide.

A parallel line scales imitation with multi-task or pre-trained VLA backbones, including RT-1 [25], RT-2 [26], OpenVLA [27], Octo [28], Open X-Embodiment [29], and π_0 [30]. Concurrent VLA-JEPA [31] predicts leakage-free future latents with a pretrained VLM and flow matching, while JEPA-VLA [32] injects video-predictive embeddings into existing VLAs. JEPA Policy instead trains on each task’s demonstrations, shares action and future tokens in one stack, uses two MIP steps, and relies on action loss against collapse.

D. World Models and Future Prediction

World and forward models predict action-conditioned futures for planning or model-predictive control [33]. V-JEPA 2-AC [34] uses a frozen video-pretrained encoder and latent predictor for image-goal MPC. CURL [35], SPR [36], DreamerV3 [37], and TD-MPC2 [38] likewise learn representations or dynamics for rollout. JEPA Policy learns no separate dynamics model, performs no planning rollout, and reconstructs no future images. It consumes one demonstrated future latent paired with the supervised action as a policy-training signal.

E. Runtime Failure Monitoring for Learned Policies

Runtime failure detection for learned policies draws on a broad literature in predictive uncertainty. Representative approaches include softmax confidence for out-of-distribution inputs [39], deep ensembles [40], and conformal prediction. The latter provides distribution-free safety margins and has been applied to planning among dynamic agents [41]. Sentinel [42]

combines temporal action consistency with a vision–language model that evaluates task progress. FAIL-Detect [43] detects failures in imitation policies without requiring failure data. Our diagnostic has a narrower scope. It is a by-product of the training objective rather than a purpose-built monitor. Because the policy already predicts a future representation, comparison with the realized representation requires one encoder pass and no additional model. We therefore present it as a per-task-calibrated risk signal, not as a competitor to dedicated monitors. A direct comparison with such monitors remains future work.

III. METHOD

This section specifies JEPA Policy’s paired objective, shared stack, two-step training rule, and deployment diagnostic.

A. Problem Formulation

Given an expert demonstration dataset

$$\mathcal{D} = \{(o_t, \mathbf{a}_{t:t+H}, o_{t+k})\}, \quad (1)$$

where o_t denotes the current observation or observation history window, $\mathbf{a}_{t:t+H}$ denotes the expert action sequence over a prediction horizon of H , and o_{t+k} denotes the future observation at horizon k . We use $\mathbf{a}_{t:t+H}$ as compact notation for the next H actions, $(\mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H})$. Both H and k are counted in environment control steps; their wall-clock meaning is quantified in Sec. IV-A. The horizon k sets how far the target observation lies in the future. If k is too small, $\mathbf{z}_{t+k}$ closely resembles the current observation and provides little new information. If k is too large, the current action chunk has little influence on the target and the pairing becomes weak. We therefore set k on the same order of magnitude as the action horizon H and determine its concrete value by ablation. Conventional direct policy learning formulates imitation learning as a supervised mapping from the current observation to expert actions:

$$\pi_\theta(o_t) \rightarrow \hat{\mathbf{a}}_{t:t+H}. \quad (2)$$

This formulation requires the policy only to predict expert actions, without explicitly constraining the future outcomes that action sequence induces in the demonstration trajectory.

JEPA Policy learns the following joint mapping:

$$\pi_\theta(o_t) \rightarrow \{\hat{\mathbf{a}}_{t:t+H}, \hat{\mathbf{z}}_{t+k}\}, \quad (3)$$

where $\hat{\mathbf{a}}_{t:t+H}$ is the predicted action sequence and $\hat{\mathbf{z}}_{t+k}$ is the predicted future observation representation. Here $\pi_\theta(o_t)$ is a shorthand for the final composite policy. For a unified notation, we write the network as the coarse-to-refined predictor $F_\theta(o_t, \mathbf{a}_{\text{in}}, \mathbf{z}_{\text{in}}, u, v)$. Its inputs are the current observation, an action input, a future-representation input, and the interval endpoints (u, v) . The action input can be zero or noisy. The network outputs an action and a future observation representation. F_θ denotes the shared Transformer policy described in Sec. III-B; the two names refer to the same network. The future observation representation target is obtained by passing the actual future observation through the observation encoder:

$$\mathbf{z}_{t+k} = E_{\text{rgb}}(o_{t+k}), \quad (4)$$

where $E_{\text{rgb}}(\cdot)$ is the image path of the same observation encoder E . It globally pools and concatenates the per-camera backbone features before they are fused with low-dimensional inputs. The conditioning representation $\mathbf{c}_t = E(o_t)$ is the output of that fusion layer and therefore also carries proprioception, whereas the future target is image-only and higher-dimensional (1024 against 384 in our configuration). For the current observation, the same image path gives $\mathbf{z}_t = E_{\text{rgb}}(o_t)$. The conditioning representation is $\mathbf{c}_t = \text{Fuse}([\mathbf{z}_t, \mathbf{s}_t])$, where $\mathbf{s}_t$ contains the low-dimensional proprioceptive inputs. Figure 2 shows $\mathbf{z}_t$ and $\mathbf{z}_{t+k}$ as outputs of the same function at two times and leaves the fusion step implicit. Both use one encoder and the same image-backbone parameters. Their values differ because they are computed from observations at different times. Unlike future image reconstruction, JEPA Policy predicts future observation representations only in latent space, avoiding the modeling of low-level visual details such as texture, illumination, and background that are weakly related to robot control decisions.

The shared encoder E is trained end to end, without a frozen or EMA target encoder. We stop gradients only through the future target $\mathbf{z}_{t+k}$; $\mathcal{L}_{\text{future}}$ still updates E through the current-observation path $\mathbf{c}_t$, as detailed in Sec. III-C.

B. Shared Action–Future Observation Representation Policy

The overall structure of JEPA Policy is shown in Fig. 2. The policy is exactly the $F_\theta(o_t, \mathbf{a}_{\text{in}}, \mathbf{z}_{\text{in}}, u, v)$ introduced in Sec. III-A, consisting of an observation encoder, H action input tokens, a single future input token, a shared Transformer, an action prediction head, and a future observation representation prediction head. The current observation is first encoded into a conditioning representation:

$$\mathbf{c}_t = E(o_t). \quad (5)$$

Together with an embedding of the two interval endpoints (u, v) of the current step, $\mathbf{c}_t$ forms the conditioning memory

$$\mathbf{M} = \text{Enc}([\text{TimeEmb}(u, v), \text{Embed}_o(\mathbf{c}_t)]), \quad (6)$$

which the shared Transformer reads by cross-attention. Separate linear projections embed the action input $\mathbf{a}_{\text{in}}$ and future-representation input $\mathbf{z}_{\text{in}}$. The resulting tokens occupy $H + 1$ positions in one sequence. A learned positional embedding $\mathbf{p}$ covers all positions, while $\mathbf{e}_z$ marks the future position:

$$\mathbf{q}_a = \text{Embed}_a(\mathbf{a}_{\text{in}}) + \mathbf{p}_{1:H}, \quad (7)$$

$$\mathbf{q}_z = \text{Embed}_z(\mathbf{z}_{\text{in}}) + \mathbf{e}_z + \mathbf{p}_{H+1}. \quad (8)$$

The step parameters are deliberately *not* added to these tokens; they enter only through $\mathbf{M}$ in Eq. (6). The shared Transformer then jointly models the two token types:

$$\mathbf{Y} = T_\theta([\mathbf{q}_a, \mathbf{q}_z], \mathbf{M}), \quad (9)$$

where T_θ denotes the shared policy Transformer (the backbone of F_θ), $[\cdot, \cdot]$ denotes token concatenation, and each of its layers applies unmasked self-attention over the $H + 1$ concatenated tokens followed by cross-attention to $\mathbf{M}$. The output $\mathbf{Y}$ is

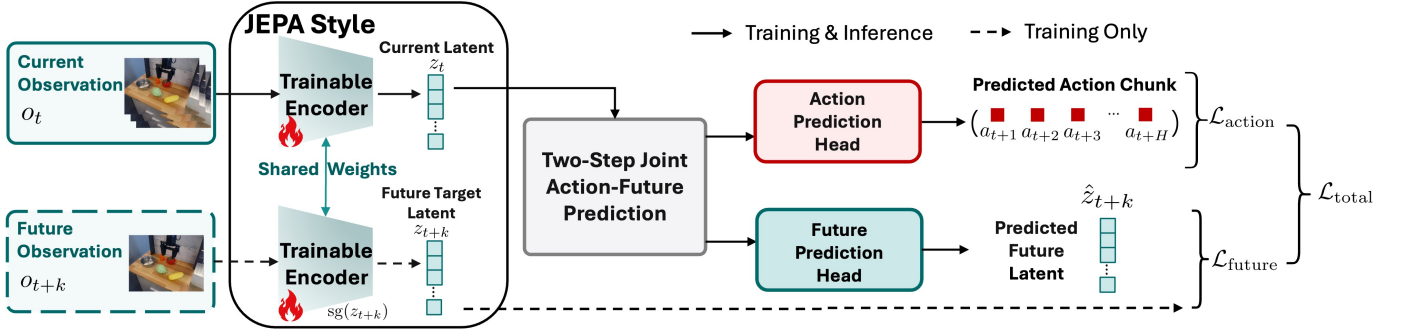


Fig. 2. Overall architecture of JEPA Policy. A shared encoder processes both observations. $\mathbf{z}_t$ and $\mathbf{z}_{t+k}$ are the outputs of its image path at the two times. The conditioning vector $\mathbf{c}_t$ fuses $\mathbf{z}_t$ with low-dimensional inputs (Sec. III-B); the fusion is omitted from the diagram. Action tokens $\mathbf{q}_a$ and future-representation tokens $\mathbf{q}_z$ enter one Transformer T_θ and interact in every self-attention layer. Separate heads produce the action sequence $\hat{\mathbf{a}}_{t:t+H}$ and future observation representation $\hat{\mathbf{z}}_{t+k}$. The same encoder produces the stopped target $\mathbf{z}_{t+k} = E_{\text{rgb}}(o_{t+k})$. Shared processing allows the future-prediction objective to update the representation used by the action head.

split into action token representations $\mathbf{Y}_a$ and future token representations $\mathbf{Y}_z$:

$$\mathbf{Y}_a, \mathbf{Y}_z = \text{split}(\mathbf{Y}). \quad (10)$$

The two prediction heads then output the action sequence and the future observation representation, respectively:

$$\hat{\mathbf{a}}_{t:t+H} = h_a(\mathbf{Y}_a), \quad (11)$$

$$\hat{\mathbf{z}}_{t+k} = h_z(\mathbf{Y}_z), \quad (12)$$

where $h_a(\cdot)$ denotes the action prediction head and $h_z(\cdot)$ denotes the future observation representation prediction head. The overall mapping is exactly $F_\theta(o_t, \mathbf{a}_{\text{in}}, \mathbf{z}_{\text{in}}, u, v) = (\hat{\mathbf{a}}_{t:t+H}, \hat{\mathbf{z}}_{t+k})$. The two steps use interval endpoints $(u, v) = (0, \tau)$ and $(\tau, 1)$. In the first step, $\mathbf{a}_{\text{in}} = \mathbf{0}$ and $\mathbf{z}_{\text{in}} = \mathbf{0}$. The tokens then reduce to learned queries formed by the input-projection biases, $\mathbf{p}$, and $\mathbf{e}_z$. This step regresses a coarse prediction from the current observation. The second step receives noised expert targets during training and the first-step predictions during inference. It then refines both outputs.

In this structure, future observation representation prediction is not an isolated auxiliary branch appended after the policy. Shared attention allows future prediction to shape the policy representation used for action generation. The action and future tokens occupy one unmasked self-attention stack and can interact in every layer. The action representation $\mathbf{Y}_a$ therefore contains information from the future token during the forward pass. The future loss also backpropagates through the same attention layers that produce the action representation. Separate-branch designs such as ACT-JEPA [6] behave differently. Their future loss can update a shared encoder, but future predictions do not enter the computation of the action tokens. The shared stack thus creates two routes from future prediction to action generation. The forward route lets action tokens attend to the future token during the policy computation. The backward route lets the future loss update the same intermediate parameters used by the action head. A separate future branch may retain the second route only through an upstream encoder, while remaining absent from the computation that produces the action tokens. This distinction motivates both the shared-versus-separate comparison and the route-cut controls in the

experiments. Section IV-D tests whether this difference in topology accounts for the performance gain.

C. Joint Action–Future Representation Objective

The action prediction loss is defined as

$$\mathcal{L}_{\text{act}} = \langle (\hat{\mathbf{a}}_{t:t+H} - \mathbf{a}_{t:t+H})^2 \rangle, \quad (13)$$

where $\langle \cdot \rangle$ denotes the mean over the action dimension, the horizon H and the batch, so that the loss scale is independent of H and of the action dimensionality. The future observation representation prediction loss aligns the predicted future representation with the target representation of the actual future observation in the demonstration:

$$\mathcal{L}_{\text{future}} = d_{\text{NMSE}}(\hat{\mathbf{z}}_{t+k}, \text{sg}(\mathbf{z}_{t+k})), \quad (14)$$

where $d_{\text{NMSE}}(\hat{\mathbf{z}}, \mathbf{z}) = \|(\hat{\mathbf{z}} - \mathbf{z}) / \max(\|\mathbf{z}\|_{\text{RMS}}, 10^{-6})\|_2^2 / d_z$, and d_z is the latent dimension. We then average over the batch. This target-normalized MSE separates the latent-space scale from the action-space scale and simplifies loss balancing (Sec. IV-A). $\text{sg}(\cdot)$ denotes a stop-gradient that we **always** apply to the future target representation $\mathbf{z}_{t+k} = E_{\text{rgb}}(o_{t+k})$. The current and future observations use the same encoder parameters E . The future loss still updates this encoder because $\hat{\mathbf{z}}_{t+k}$ depends on $\mathbf{c}_t = E(o_t)$. Gradients therefore reach E through the current-observation path. The stop-gradient cuts only the future-observation target path $\mathbf{z}_{t+k} = E_{\text{rgb}}(o_{t+k})$, preventing direct target-side co-adaptation but not, by itself, excluding a constant representation. The future loss still shapes the encoder through the current-observation path, while the action loss $\mathcal{L}_{\text{act}}$ supplies the task-driven constraint tested in Sec. IV-I. In implementation, both the action loss and the future observation representation loss are averaged over the batch and output dimensions.

The overall training objective is

$$\mathcal{L} = \lambda_a \mathcal{L}_{\text{act}} + \lambda_z \mathcal{L}_{\text{future}}, \quad (15)$$

The coefficients λ_a and λ_z weight these two terms.

Two settings of λ_z are implemented. Fixed mode uses a constant and appears only where stated. Unless otherwise

stated, the main experiments use *adaptive ratio* mode, which recomputes λ_z once per minibatch:

$$\lambda_z = \text{clip}\left(\rho \frac{\text{sg}(\lambda_a \mathcal{L}_{\text{act}})}{\text{sg}(\mathcal{L}_{\text{future}}) + \varepsilon}, \lambda_{\min}, \lambda_{\max}\right), \quad (16)$$

For the two-step objective, the losses in Eq. (16) sum the unweighted, interval-scaled components from both steps; one λ_z is shared by both. Stop-gradient applies only inside Eq. (16), so λ_z carries no gradient; both terms in Eq. (15) retain their normal gradients. We set $\varepsilon = 10^{-8}$ and, in adaptive-ratio experiments, $\lambda_a = 1$, default $\rho = 0.10$, and $[\lambda_{\min}, \lambda_{\max}] = [10^{-6}, 0.1]$. We ablate $\rho \in \{0.05, 0.10, 0.20\}$ in Sec. IV-F. Without clipping, $(\lambda_z \mathcal{L}_{\text{future}})/(\lambda_a \mathcal{L}_{\text{act}}) \approx \rho$. The upper and lower bounds can move this ratio below and above ρ , respectively. Thus, ρ is neither an exact constraint nor the future term's share of total loss.

Through Eq. (15), action prediction and future observation representation prediction are jointly optimized in the same policy network, introducing paired action–future supervision for diffusion-free imitation learning.

D. MIP-Style Two-Step Prediction Training

To avoid the inference overhead of the long-chain multi-step denoising in diffusion policies, JEPA Policy adopts a MIP-style two-step prediction process. MIP uses **supervised iterative computation** and **stochasticity injection**. Both prediction steps are supervised, and the second-step input contains noise. MIP does not learn a continuous velocity field or require multi-step integration. Our MIP-style implementation directly regresses the target at both steps and scales the two residuals by their interval lengths. The target can be an action or a future observation representation. We apply this process simultaneously to the expert action sequence and the future observation representation. The two-step interpolation parameter τ is a fixed hyperparameter ($\tau \in (0, 1)$, default $\tau = 0.9$) that takes the same value during training and inference, and it is unrelated to the time-step index t used in subscripts such as o_t and $\mathbf{a}_{t:t+H}$.

The first step takes zero vectors as input at time index 0 and directly produces coarse actions and coarse future observation representations from the current observation:

$$(\hat{\mathbf{a}}_{t:t+H}^{(0)}, \hat{\mathbf{z}}_{t+k}^{(0)}) = F_{\theta}(o_t, \mathbf{0}, \mathbf{0}, 0, \tau). \quad (17)$$

The first-step loss is

$$\mathcal{L}^{(0)} = \lambda_a \left\langle \left(\frac{\hat{\mathbf{a}}_{t:t+H}^{(0)} - \mathbf{a}_{t:t+H}}{\tau} \right)^2 \right\rangle + \frac{\lambda_z}{\tau^2} d_{\text{NMSE}}\left(\hat{\mathbf{z}}_{t+k}^{(0)}, \text{sg}(\mathbf{z}_{t+k})\right). \quad (18)$$

The second step constructs noisy inputs near the expert target. Given Gaussian noise $\epsilon_a, \epsilon_z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, define

$$\tilde{\mathbf{a}}_{t:t+H} = \mathbf{a}_{t:t+H} + (1 - \tau)\epsilon_a, \quad (19)$$

$$\tilde{\mathbf{z}}_{t+k} = \text{sg}(\mathbf{z}_{t+k}) + (1 - \tau)\epsilon_z. \quad (20)$$

The model then refines both inputs back to the expert action and the true future observation representation at time index τ :

$$(\hat{\mathbf{a}}_{t:t+H}^{(1)}, \hat{\mathbf{z}}_{t+k}^{(1)}) = F_{\theta}(o_t, \tilde{\mathbf{a}}_{t:t+H}, \tilde{\mathbf{z}}_{t+k}, \tau, 1). \quad (21)$$

Two-Step Joint Action-Future Prediction

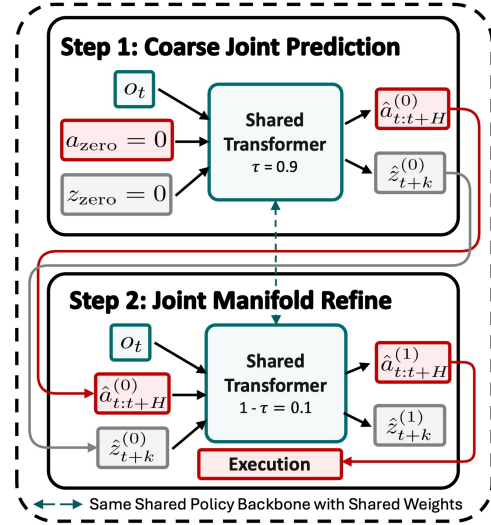


Fig. 3. Coarse-to-refined MIP-style training for joint action and future observation representation prediction. The first step maps a zero input (time 0) to a coarse prediction, and the second refines it (time τ). The diagram shows inference, where the second step receives the first-step output. During training, it instead receives the noised expert target from Sec. III-D. Both steps are supervised, with residuals scaled by $\frac{1}{\tau}$ and $\frac{1}{1-\tau}$. The method uses no diffusion-style iterative sampling.

The second-step loss is

$$\mathcal{L}^{(1)} = \lambda_a \left\langle \left(\frac{\hat{\mathbf{a}}_{t:t+H}^{(1)} - \mathbf{a}_{t:t+H}}{1-\tau} \right)^2 \right\rangle + \frac{\lambda_z}{(1-\tau)^2} d_{\text{NMSE}}\left(\hat{\mathbf{z}}_{t+k}^{(1)}, \text{sg}(\mathbf{z}_{t+k})\right). \quad (22)$$

The two-step residuals are divided by τ and $1 - \tau$ to place them on a common scale. The second-step input differs from the expert target by $(1 - \tau)\epsilon$. Division by $1 - \tau$ therefore yields a unit-scale correction. The first-step input is zero rather than a perturbed target, so the same derivation does not apply to that step. Its division by τ is the matching convention for an interval of length τ and should not be interpreted as correcting injected noise. The two steps are decoupled during training: the input of the second step is constructed from the ground-truth target and noise, not from the output of the first step. The two supervised prediction steps (iterative computation) and the input noise of the second step (stochasticity injection) together constitute the two effective ingredients of MIP. The final joint MIP objective is

$$\mathcal{L}_{\text{MIP}} = \mathbb{E}_{\mathcal{D}, \epsilon_a, \epsilon_z} [\mathcal{L}^{(0)} + \mathcal{L}^{(1)}]. \quad (23)$$

The expectation is over training tuples and both noise variables. We use the simplified second-step input $\mathbf{a}_{t:t+H} + (1 - \tau)\epsilon_a$ rather than the original $\tau \mathbf{a}_{t:t+H} + (1 - \tau)\epsilon_a$; the two coincide as τ approaches 1.

Fig. 3 illustrates the two-step MIP-style joint training procedure. At inference, JEPA Policy first evaluates the zero-input coarse step above with the same fixed τ , then supplies both coarse outputs to a noise-free refinement:

$$(\hat{\mathbf{a}}_{t:t+H}^{(1)}, \hat{\mathbf{z}}_{t+k}^{(1)}) = F_{\theta}(o_t, \hat{\mathbf{a}}_{t:t+H}^{(0)}, \hat{\mathbf{z}}_{t+k}^{(0)}, \tau, 1). \quad (24)$$

The refined action $\hat{\mathbf{a}}_{t:t+H}^{(1)}$ controls the robot. In the original MIP formulation, the second-step input is the first-step output scaled by τ , i.e., $\tau \hat{\mathbf{a}}_{t:t+H}^{(0)}$; we adopt the simplified form above, and the two coincide as τ approaches 1.

E. Future Observation Representation Prediction under Paired Action Supervision

Predicting a future observation representation from the current observation alone is under-constrained. Similar observations may precede different futures, so plain regression may average targets or encode control-irrelevant variation.

JEPA Policy instead uses the demonstration tuple of Sec. III-A: the action target and $\mathbf{z}_{t+k}$ from the same segment jointly supervise the two outputs. At the supervision level, each segment supplies both labels; at the representation level, the future loss reaches the action representation through the shared attention stack. Each training sample therefore preserves the correspondence between one demonstrated action and its observed outcome. The predictor remains deterministic and can still average across ambiguous repeated observations. This construction neither assumes a one-to-one action–future mapping nor recovers every possible future mode. It uses paired evidence more fully while retaining the limitations of deterministic regression. The future branch shapes the policy during training and supplies the diagnostic defined next for deployment-time use.

F. Deployment-Time Future-Consistency Diagnostics

At inference, JEPA Policy outputs the refined control action sequence together with the refined future observation representation $\hat{\mathbf{z}}_{t+k}^{(1)}$. After the policy executes the action, the actual future observation $\mathbf{o}_{t+k}^{\text{real}}$ becomes available, and the same observation encoder produces its representation:

$$\mathbf{z}_{t+k}^{\text{real}} = E_{\text{rgb}}(\mathbf{o}_{t+k}^{\text{real}}). \quad (25)$$

We accordingly define the rollout-level future-consistency error:

$$e_{\text{fc}}(t, k) = d_{\text{NMSE}}\left(\hat{\mathbf{z}}_{t+k}^{(1)}, \text{sg}\left(\mathbf{z}_{t+k}^{\text{real}}\right)\right). \quad (26)$$

When policy behavior remains close to the demonstrations, the predicted representation should agree with the observed post-execution representation. A large e_{fc} may instead indicate an unexpected behavior, an abnormal contact outcome, or a visual state outside the training distribution. We treat this quantity as a diagnostic signal for failure analysis and human-intervention triggering, not as a strict probabilistic uncertainty estimate. In practical deployment, the alarm threshold can be chosen using a sliding-window average $\bar{e}_{\text{fc}}$, task-phase-conditioned thresholds, or ROC curves computed from successful/failed validation rollouts. The signal becomes available only after the corresponding future observation has been reached, so it diagnoses the preceding prediction rather than preventing that action from executing. Its operational use is to inform the next control decision, request intervention, or annotate a rollout for later analysis. Section IV-J therefore evaluates ranking and calibration separately instead of treating the raw error as an instantaneous safety guarantee.

IV. EXPERIMENTS

We evaluate JEPA Policy on nine simulated manipulation tasks from three benchmark suites, using three training seeds per configuration. The evaluation also includes a pre-specified collapse audit on seven tasks over 84 checkpoints (Sec. IV-I) and a 5,200-episode diagnostic audit on the original eight-task set (Sec. IV-J). Sec. IV-K repeats the comparison on physical hardware.

A. Setup

Tasks. We use tasks from LIBERO [44], robomimic [8], and MimicGen [45]. These suites are built on robo-suite/MuJoCo [46], [47]. LIBERO provides MokaMoka and MugMug, with 50 demonstrations and 20,794/12,909 transitions. Robomimic provides Square PH, Tool Hang PH, and four-camera Transport PH, with 200 demonstrations and 30,154/95,962/93,752 transitions. MimicGen provides Coffee Preparation D1, Kitchen D1, Three Piece Assembly D1, and Hammer Cleanup D1, each with 1,000 demonstrations and 286,847 to 687,674 transitions. The four-camera Transport dataset (four 84×84 RGB views) was generated for this work and passed an independent structural audit; a two-camera variant exists for one seed and is excluded from the main comparison.

Protocol. The primary MIP, Diffusion Policy, and JEPA Policy runs train for 300k gradient steps. We use batch size 256, learning rate 10^{-4} , and EMA rate 0.995. Each run is evaluated every 10k steps under a fixed evaluation seed, giving 30 rollout evaluations. We report the maximum success rate over the 30 evaluations (best-checkpoint protocol), averaged over seeds {41, 42, 43}. The same checkpoint rule and rollout budget are applied to every primary arm, so selection optimism affects their absolute levels in the same direction. Table IV separately reports three selection-free summaries to test whether the difference between JEPA Policy and MIP depends on this rule. The controlled topology and route-cut comparisons on Transport use an aligned 10k–230k window (23 evaluations); all reported maxima for those comparisons occur within this window. The reported success rates have finite rollout resolution; differences corresponding to only one or two episode outcomes should not be over-interpreted.

Timing. All simulated environments run at 20 Hz control. The policy predicts $H=10$ actions on robomimic and MimicGen and $H=16$ on LIBERO. In both cases, it executes the first eight actions before replanning. Actions run at 20 Hz, and the policy produces a new chunk at 2.5 Hz. The future horizon k counts environment transitions after the newest observation frame: the default $k=4$ target lies 0.2 s ahead ($k=2/6/8$: 0.1/0.3/0.4 s), i.e., within the 0.4 s duration of the currently executed action chunk itself.

Implementation details. The observation encoder E is a ResNet-18 [48] image encoder trained jointly with the policy by default; a frozen DINOv2 [49] ViT-S/14 encoder (CLS token) is supported as an alternative backbone. The future target is a single pooled global embedding rather than a sequence of patch tokens. It uses only the image channels at the input of the encoder’s fusion layer, excluding low-dimensional inputs such

as proprioception. We apply stop-gradient to this target. To preserve the spatial correspondence between current and future observations under image augmentation, the two share the same random crop window during training (temporal-consistent crop). The future loss uses target-RMS-normalized MSE. We set the interpolation parameter to $\tau = 0.9$ and the default future horizon to $k = 4$. Inference uses the EMA weights. This exponential moving average is Polyak averaging of the trained policy for evaluation and is applied to all methods, including the baseline. It is not a BYOL-style EMA *target* encoder. JEPA Policy produces its future target with the live shared encoder and applies stop-gradient (Sec. III-B). In the main experiments, the action loss weight is $\lambda_a = 1$, and the future loss weight follows Eq. (16) with target ratio $\rho = 0.10$ and clip bounds $[10^{-6}, 0.1]$. We ablate $\rho \in \{0.05, 0.10, 0.20\}$. Peak device memory remains below 6 GiB per training run. On Tool Hang, JEPA Policy allocates 5.28 GiB and reserves 5.71 GiB, compared with 5.02 GiB allocated by MIP. Peak inference activations are 362/355 MiB for JEPA Policy/MIP (Table III).

B. Compared Methods

Table I contrasts the five arms in few-step inference, future-representation supervision, shared attention, and rollout diagnostics. Implementation details follow.

- **MIP (action-only)**: the minimal iterative policy of [4], run from the released implementation that this work extends. Setting $\lambda_z = 0$ and removing the future tokens recovers MIP exactly. This arm is therefore both a published baseline and an exact ablation of JEPA Policy. The architecture, optimizer, data pipeline, and evaluation harness are shared by construction.
- **Diffusion Policy** [3]: the diffusion *transformer* policy with a 10-step action horizon. This choice matches the JEPA Policy action-chunk length on the robomimic and MimicGen tasks. The aligned protocol uses the same tasks and seeds, 300k gradient steps, batch size 256, and 30 rollout evaluations under the same task-specific budget. We apply the same best-checkpoint rule to its `test/mean_score`. The sampler uses the 100-step DDPM configuration on which it was trained. For each task, we report the better of the absolute and delta action parameterizations because the absolute-action variant failed to learn three robomimic tasks. On the two LIBERO tasks, JEPA Policy uses a 16-step horizon, so the horizon match is exact only outside LIBERO. Diffusion Policy is weakest on Tool Hang (46.7), which robomimic labels as a high-precision task. On this task, accumulated pose error can affect the final insertion.
- **Dual-independent (ACT-JEPA-style topology)**: the shared conditioning path contains the observation embedding, condition encoder, future-memory adapter, and two interval embeddings (1.40 M parameters). It produces a fixed observation memory for two parameter-independent decoder stacks. The action stack has 18.93 M parameters and the future stack has 4.75 M; both use pre-norm self-attention, fixed-memory cross-attention, and an FFN. No

path connects the stacks, so each loss produces zero gradient on the other stack. Both losses still train the shared conditioning path because it has no stop-gradient. This configuration isolates the separate-branch property of ACT-JEPA. To avoid confounding topology with other design choices, it retains our visual target, stopped-target action supervision, and training recipe instead of reproducing the full ACT-JEPA method.

- **Dual-cross**: identical to Dual-independent plus a bidirectional cross-attention block at every layer (5.35 M). Each layer applies self-attention on both branches, exchanges information through the cross block, then applies the memory-attention and FFN steps. The two branches are therefore mutually visible, but through an explicit cross module rather than by sharing one attention stack.
- For reference, JEPA Policy itself passes the concatenation of action tokens $\mathbf{q}_a$ and future tokens $\mathbf{q}_z$ through a single stack (20.27 M) under full bidirectional self-attention with no causal, token-type or memory mask. All three variants share the encoder (22.90 M), the losses, the MIP two-step schedule, the stop-gradient target and $\rho = 0.10$; totals are 43.96 M (JEPA Policy), 48.38 M (Dual-independent) and 53.73 M (Dual-cross).
- **SIGReg variant**: JEPA Policy plus an explicit isotropic-Gaussian regularizer on the encoder latents in the style of SIGReg [50] (weight 0.09), used as an explicit-regularization reference in the collapse analysis.
- $\lambda_a = 0$ **variant**: JEPA Policy with the action loss weight set to zero, so the future objective trains the shared encoder alone; the anti-collapse control of Sec. IV-I. It is trained on Square PH (seed 42) under an otherwise identical recipe and audited at its 220k-step EMA checkpoint.

TABLE I
METHOD PROPERTIES OF THE COMPARED APPROACHES.

Method	Few-step inference	Future repr. supervision	Shared attention	Diagnostic signal
MIP (action-only)	Yes (two-step)	No	N/A	No
Diffusion Policy	No	No	N/A	No
Dual-independent [†]	Yes (two-step)	Yes (separate)	No	Optional
Dual-cross	Yes (two-step)	Yes (cross-attn)	No	Optional
JEPA Policy	Yes (two-step)	Yes (shared)	Yes	Yes

[†] ACT-JEPA-style separate-branch topology; a controlled proxy rather than a faithful ACT-JEPA reproduction.

C. Main Results

Table II and Fig. 4 present the main comparison on the nine LIBERO/robomimic/MimicGen tasks. JEPA Policy improves over the MIP (action-only) arm on all nine tasks. The largest gains occur on Coffee Preparation (57.5 $\rightarrow$ 77.5) and Tool Hang (65.8 $\rightarrow$ 75.0), and the nine-task mean improves by +5.6 points (77.4 $\rightarrow$ 83.0). The Coffee Preparation baseline mean is depressed by an outlier seed (seed 42 reaches only 30.0 while seeds 41/43 reach 70.0/72.5); the JEPA Policy gain remains clear on every individual seed. On Square, MugMug, Transport, and Hammer Cleanup, the gains are small relative to the evaluation granularity. On Hammer Cleanup, JEPA Policy

TABLE II
BEST-CHECKPOINT SUCCESS RATE (%) ON THE NINE TASKS.

Task	MIP (action-only)	Dual-indep.	Dual-cross	Diffusion Policy	JEPA Policy
MokaMoka	86.7 85/90/85	85.8 82.5/87.5/87.5	82.5 77.5/87.5/82.5	84.2 85/77.5/90	92.5 92.5/95/90
MugMug	94.2 97.5/92.5/92.5	90.8 87.5/90/95	86.7 90/85/85	90.0 90/87.5/92.5	96.7 97.5/95/97.5
Square	94.2 95/92.5/95	94.2 95/92.5/95	94.2 97.5/95/90	93.3 92.5/95/92.5	95.8 95/97.5/95
Tool Hang	65.8 72.5/65/60	59.2 75/52.5/50	68.3 60/65/80	46.7 47.5/52.5/40	75.0 82.5/65/77.5
Transport (4cam)	97.5 97.5/97.5/97.5	99.2 97.5/100/100	97.5 97.5/97.5/97.5	96.7 95/95/100	98.3 97.5/97.5/100
Coffee Preparation	57.5 70/30/72.5	57.5 62.5/47.5/62.5	61.7 42.5/67.5/75	66.7 72.5/60/67.5	77.5 77.5/70/85
Kitchen	68.3 67.5/62.5/75	72.5 70/75/72.5	71.7 72.5/67.5/75	69.2 70/67.5/70	73.3 72.5/77.5/70
Three Piece Assembly	60.0 57.5/60/62.5	59.2 65/52.5/60	63.3 62.5/67.5/60	59.2 65/57.5/55	64.2 67.5/65/60
Hammer Cleanup	72.7 76/68/74	68.7 68/72/66	72.7 74/74/70	70.0 72/68/70	74.0 74/70/78
Mean (9 tasks)	77.4 79.8/73.1/79.3	76.3 78.1/74.4/76.5	77.6 74.9/78.5/79.4	75.1 76.6/73.4/75.3	83.0 84.1/81.4/83.7

Large number: mean over seeds {41, 42, 43}. Small numbers: individual seeds in the same order. Matching seed indices use the same data ordering and initialization stream across arms. Bold values: best mean per task. The last row reports the nine-task mean and its per-seed values; means and gains are computed before rounding. The Transport topology cells use the aligned evaluation window from Sec. IV-A.

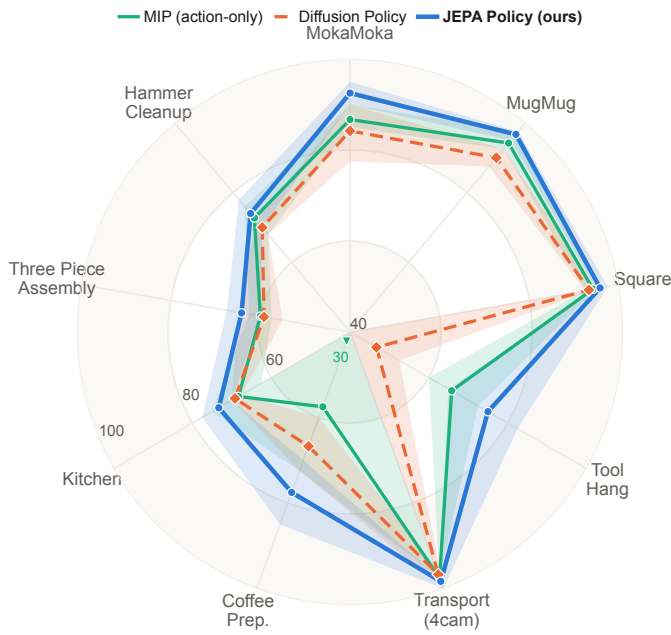


Fig. 4. Main comparison over nine tasks (line: three-seed mean best-checkpoint success; band: seed range; radial axis 40–100; means in Table II). A ▼ and printed value mark a seed below the axis floor.

reaches 74.0, compared with 72.7 for MIP, 70.0 for Diffusion Policy, and 68.7/72.7 for the dual controls. The small numbers in Table II provide the aggregate result for each seed. The weakest JEPA Policy seed (81.4 across tasks) remains above the strongest seed from every other arm (79.8 MIP, 78.1 Dual-independent, 79.4 Dual-cross, and 76.6 Diffusion Policy). This seed ordering shows that the nine-task mean is not produced by one favorable initialization of JEPA Policy or one unfavorable initialization of a comparison arm. It does not turn three seeds into independent task replicates, which is why the statistical analysis below continues to use the task as its unit.

Statistical reading. We treat the task as the unit of analysis, since the three seeds of a task are not independent of one another. Across the nine tasks, the mean gain over MIP is +5.61 points. The paired t interval is $[+0.99, +10.23]$ ($t(8) = 2.80$), and the paired bootstrap interval is $[+2.56, +9.82]$ (20,000 resamples). All nine task differences are positive, giving $p =$

0.0039 for both the exact two-sided sign test and Wilcoxon signed-rank test. Against Dual-cross, the mean is +5.43 points with an interval of $[+1.23, +9.63]$. All nine differences are positive (sign test and Wilcoxon $p = 0.0039$). Against Dual-independent, the mean is +6.70 points with an interval of $[+1.39, +12.02]$. Eight differences are positive and Transport is the exception (sign test $p = 0.039$, Wilcoxon $p = 0.012$). We do not report a seed-level test. With three seeds, the paired statistic is $t(2) = 4.18$ against a critical value of 4.303. No three-seed design can reach $p < 0.05$ under an exact two-sided sign-flip permutation, whose p -value floor is 0.25. The evidence for the effect is therefore its consistency across tasks, not its significance across seeds; Fig. 5 shows that it is also consistent across training time.

The gain does not depend on the best-checkpoint protocol.

Reporting the maximum over 30 evaluations is optimistic by construction, and the concern it raises is whether the reported gain is an artifact of that selection. We therefore recomputed the same comparison from the stored evaluation curves under three protocols that involve no selection at all (Table IV). The absolute success rates fall sharply under last-checkpoint reporting, with the nine-task MIP mean decreasing from 77.4 to 62.5. This confirms that best-checkpoint reporting is optimistic in *level*. The gain, however, remains between +4.7 and +5.6 points because both arms are selected by the identical rule and the selection bias largely cancels in the difference. The last-checkpoint row uses one evaluation per task with no temporal averaging. Its per-task sign pattern drops to five of nine, whereas both averaging protocols retain eight of nine. We retain best-checkpoint numbers in the main tables for comparison with the literature. Table IV shows that the conclusion does not depend on this reporting choice.

Against Diffusion Policy, JEPA Policy is ahead on all nine tasks, with a nine-task mean of 83.0 versus 75.1. Tool Hang accounts for much of this margin because Diffusion Policy reaches only 46.7 on that task. Excluding Tool Hang gives a more conservative mean difference of +5.4 points. The methods are closest on Transport (98.3 vs. 96.7), Square (95.8 vs. 93.3), and Hammer Cleanup (74.0 vs. 70.0). The comparison therefore supports two distinct readings. Success is comparable or better under the evaluated task and training

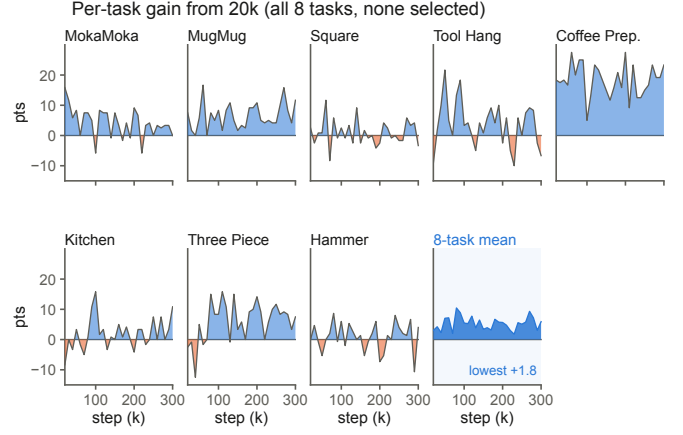


Fig. 5. Training curves for eight tasks with three completed seeds in all four arms. *Left*: eight-task mean across 30 evaluations (line: seed mean; band: seed range). *Right*: per-task JEPA Policy gain over MIP from the second evaluation onward; blue/orange denotes positive/negative gain on a shared scale. The eight-task mean is positive at all 29 evaluations (minimum +1.8); individual-task differences span -12.5 to $+27.5$ points.

TABLE III

Inference COST OF ONE POLICY DECISION (TOOL HANG, SEED 42, EMA WEIGHTS, BATCH SIZE 1) ON A PPU-ZW810E ACCELERATOR.

Method	Params (M)	Passes	Mean (ms)	p_{95} (ms)	Max Hz
MIP (action-only)	43.2	2	12.9	13.4	77.3
Diffusion Policy	42.6	100	439.5	462.6	2.3
JEPA Policy	44.0	2	13.2	14.5	75.6

Eager mode. JEPA Policy and MIP are timed interleaved in one PyTorch 2.6 process after 30 warmup and 300 synchronized iterations per arm; peak activation is 362/355 MiB (JEPA/MIP). Instrumented entry points verify “Passes.” The full audit path, including the second future projection, costs 12.8 ms. Diffusion Policy uses the same task, device, and aligned-protocol checkpoint in its trained 100-step DDPM configuration, but runs in a separate PyTorch 2.9 process. This increases uncertainty in its absolute latency, not the $33\times$ gap. The parameter counts are capacity-matched.

TABLE IV

SENSITIVITY OF THE NINE-TASK MEAN SUCCESS RATE (%) TO THE CHECKPOINT REPORTING PROTOCOL.

Protocol	Baseline	JEPA	Gain	Improved
Best of 30 evaluations	77.4	83.0	+5.6	9/9
Mean of last 5 evaluations	63.1	68.4	+5.3	8/9
Mean of all 30 evaluations	62.8	67.5	+4.7	8/9
Last checkpoint only	62.5	67.5	+5.1	5/9

Recomputed from the stored 30-evaluation curves of the same runs ($n = 3$ seeds). “Improved” counts tasks with a positive three-seed mean gain. Values and gains are computed before rounding. The first row reproduces Table II.

configurations, but the magnitude of the average gain is task-dependent. The latency comparison is more uniform because it follows directly from two policy passes rather than a 100-step denoising chain. **Inference cost.** Table III measures hardware latency and verifies the two forward passes; the second control pass skips the future projection. JEPA Policy averages 13.2 ms ($p_{95} = 14.5$ ms), using 3% of the 400 ms budget and sustaining 76 Hz. Relative to MIP, the future branch adds 0.79 M parameters, 0.29 ms (+2.2%), and 7 MiB of peak activation memory. The trained 100-step Diffusion Policy uses 100 verified calls and takes 439.5 ms (2.3 Hz), or $33\times$ longer, at a similar parameter count. Shortening it to 50/20/10/4 steps

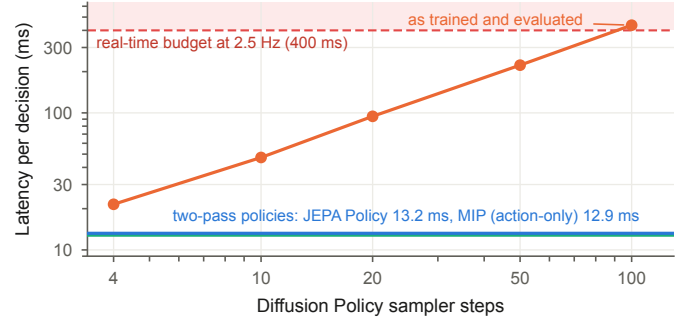


Fig. 6. Decision latency against Diffusion Policy sampler length (Tool Hang, same accelerator, batch 1, log-log). Shortened orange chains are timing-only; shading marks latency above the 400 ms budget. JEPA Policy and MIP form one blue band (13.2 vs. 12.9 ms).

gives 223.0/94.4/47.3/21.5 ms (Fig. 6); these are timing-only settings, and four steps remain $1.6\times$ slower. Per-call costs are comparable (≈ 4.4 ms per denoising step and ≈ 6.6 ms per JEPA Policy pass); the difference is the call count. These single-task, single-seed timings establish only the latency scale.

Training and evaluation cost. Training is where the future branch incurs cost. For the same 300,000 steps, it raises optimization time by 12–22% ($0.222 \rightarrow 0.271$ s/step on Tool Hang). The 30 evaluations add 2.7–5.2 h, up to one fifth of a run (Table V). A JEPA Policy step costs about $1.8\times$ a Diffusion Policy step (median over the original eight-task timing audit; $1.5\text{--}2.1\times$ on the three shown) because it performs two passes and encodes the future. This higher optimization cost accompanies a $33\times$ cheaper decision. At one seed, JEPA Policy reaches a shared success target in fewer steps than each comparison arm on six of the eight tasks in this timing-to-target audit, but the direction is not significant ($p = 0.29$). Every run uses one accelerator, takes about one day, and requires no video pretraining. The dual variants were about twice as slow in a different launch batch, so that observation is uncontrolled.

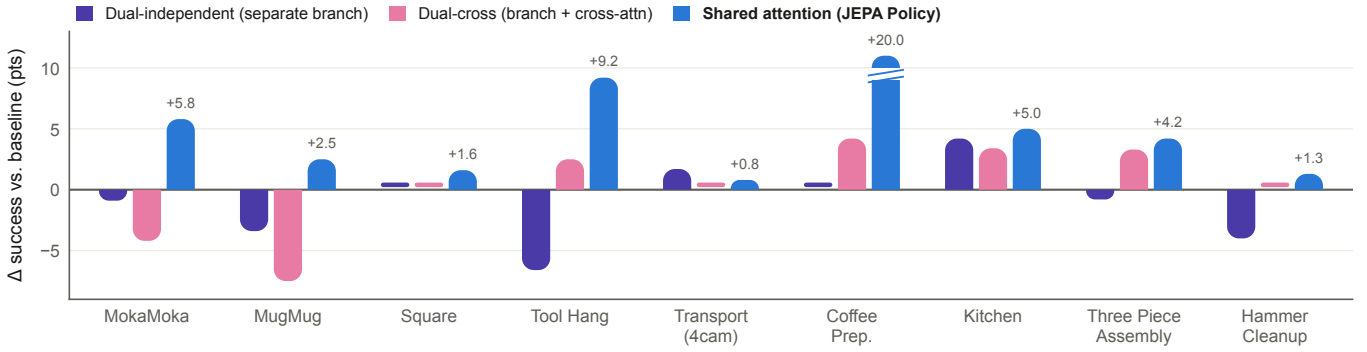


Fig. 7. Gain over MIP (three-seed mean best-checkpoint difference; per-task means in Table II). Dashes mark ties; the broken Coffee Preparation bar is +20.0; Transport topology values use the aligned window defined in Sec. IV-A. JEPA Policy averages +5.6 points across nine improvements, versus -1.1 and +0.2 for the dual variants.

TABLE V
TRAINING AND EVALUATION COST OF ONE RUN (300,000 GRADIENT STEPS, SEED 42) ON THE ACCELERATOR OF TABLE III.

Task	Arm	s/step	Train (h)	Eval (h)	Total (h)
Tool Hang	Diffusion Policy	0.136	11.3	N/A	N/A
	MIP (action-only)	0.222	18.5	4.8	23.3
	JEPA Policy	0.271	22.6	4.8	27.4
Square	Diffusion Policy	0.136	11.4	N/A	N/A
	MIP (action-only)	0.236	19.7	2.7	22.4
	JEPA Policy	0.289	24.0	2.7	26.7
MokaMoka	Diffusion Policy	0.206	17.2	N/A	N/A
	MIP (action-only)	0.273	22.7	5.2	27.9
	JEPA Policy	0.305	25.5	5.1	30.6

“Train” is run wall-clock time minus the 30 periodic rollout evaluations and therefore measures optimization only. “Eval” is the subtracted evaluation time, and “Total” is their sum. “s/step” is training time divided by 300,000. The Diffusion Policy logger records per-step optimization time but not evaluation duration, so its Eval and Total entries are marked N/A. Because it uses a different codebase, its s/step is indicative rather than a controlled measurement.

D. Shared Attention vs. Separate Branches

We test whether future prediction must share the action-generating stack to improve the policy. Figure 7 compares JEPA Policy with two dual-branch implementations that retain the training recipe, losses, and parameter protocol while changing only the topology. Dual-independent places the separate-branch topology of ACT-JEPA [6] within our training recipe. It retains the same losses and MIP training schedule, while using stopped targets instead of an EMA target encoder. It is therefore a controlled proxy for the separate-branch *design choice*, not a faithful reproduction of ACT-JEPA. We remove differences in the original objective and EMA machinery so that the comparison isolates topology.

Routing the future loss through a separate branch does not reproduce the gain. Averaged over the nine tasks, Dual-independent reaches 76.3 and Dual-cross reaches 77.6, compared with 77.4 for MIP and 83.0 for JEPA Policy. The two dual variants remain close to MIP, while the shared stack is 5.4–6.7 points higher than all three. JEPA Policy is the sole best variant on eight tasks. The exception is Transport (4cam), where Dual-independent reaches 99.2 and JEPA Policy reaches 98.3. This difference is smaller than one evaluation episode (2.5 points) in the aligned 10k–230k window. At the

task level, Dual-independent is at or below MIP on seven of nine tasks and Dual-cross on five of nine. The separate-branch variants therefore match the baseline overall. Adding the future loss outside the shared stack yields no mean improvement, and cross-attention between the branches does not recover the gain. Two properties help interpret this result. First, both have *more* trainable parameters than JEPA Policy (48.38 M and 53.73 M against 43.96 M, Sec. IV-B) and neither beats the MIP (action-only) arm, so the gain cannot be attributed to added capacity. Second, Dual-cross gives the two token groups mutual visibility through cross-attention at every layer but still provides no gain. Mutual visibility alone is therefore insufficient. JEPA Policy also uses the same parameters to produce both readouts. Section IV-E measures how this difference changes the gradient reaching the action pathway. These results identify joint processing of action and future tokens in one Transformer as the important architectural component. A future-prediction branch outside the shared stack does not provide the same effect. This conclusion is stronger than observing that JEPA Policy has an additional loss. The dual controls retain that loss, use more parameters, and expose both branches to the same observation memory, yet their mean performance remains near MIP. The controlled difference is where the future objective enters the action-generating computation. The aligned Transport window preserves every reported maximum in this comparison; excluding Transport entirely also leaves the ordering unchanged.

E. Where the Future Objective Acts

The ablation above shows *that* the topology matters; this subsection reports what the three topologies do to the gradient, which is the only channel through which the future objective can change the action parameters. We record, at every logged training step, the norm of the future-loss gradient on the parameters that produce the action readout, relative to the action-loss gradient on those same parameters.

Three findings distinguish the arms (Fig. 8). In Dual-independent, the ratio is zero at every step and seed because the decoder stacks share no parameters. The future loss still reaches the shared conditioning path, so this result concerns the action decoder rather than the entire representation. This

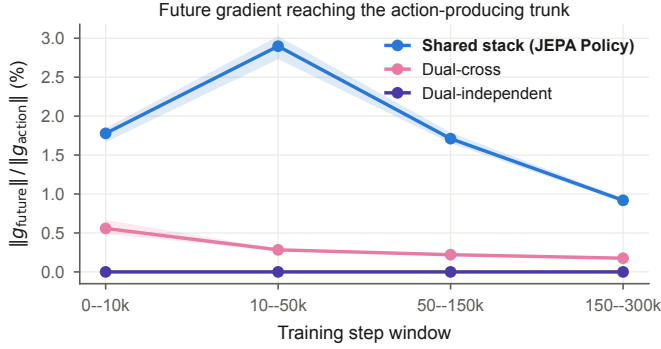


Fig. 8. Norm of the future-loss gradient reaching the trunk that produces the action, relative to the action-loss gradient on the same parameters, averaged over training windows (line: mean over seeds {41, 42, 43}; band: seed range; Tool Hang). In Dual-independent the quantity is exactly zero at every logged step, by construction. The shared stack stays an order of magnitude above Dual-cross throughout, peaking in mid-training rather than decaying.

property makes Dual-independent a direct control for topology. In Dual-cross the ratio is non-zero but small, and it decays monotonically from 0.5% to 0.2%; cross-attention does connect the branches, but the connection thins out as training proceeds. In the shared stack, the ratio rises to 2.9% in the 10-50k window and remains 0.9% at 300k. It is five to ten times the Dual-cross ratio in every window, with little variation among seeds. At convergence the same ordering holds on a fixed evaluation manifest: 0.68% for the shared stack against 0.135% for Dual-cross and exactly 0 for Dual-independent.

Two observations limit this interpretation. First, mutual visibility alone does not explain the ranking. The action-loss gradient on the Dual-cross future tokens is about five times larger than in the shared stack, yet Dual-cross shows no gain. The relevant direction is therefore the future objective reaching the action parameters, not the reverse. Second, we do not report a gradient *alignment* statistic. The cosine between the two gradients is small ($|\cos| < 0.03$) and its sign is not stable across measurement conditions, so we treat only the magnitude as evidence. The measurement is a share of gradient norm, not of the optimizer’s update. AdamW can amplify or attenuate a persistent component, so these values should not be read as update shares.

Cutting each route. The gradient shares above are correlational: they show that the future objective reaches the action parameters, not that the gain depends on it. We therefore severed each route directly and ran a matched training and evaluation comparison at one seed (Table VI). Severing the backward route removes the aggregate gain. Back-cut averages 73.1, compared with 73.8 for MIP at the same seed and 82.8 for the full method. A future head whose gradient cannot reach the shared trunk therefore provides no aggregate improvement. Severing the forward route retains about half of the gain (78.4). Together with Fig. 8, this experiment shows where the future objective acts. Most of the gain depends on its gradient reaching the shared representation. Direct attention from action tokens to the future token provides the remaining improvement.

This probe has two limitations. It uses one seed, so it supports only the aggregate ordering rather than a stable effect size. Each 40-episode evaluation also has a 2.5-point resolution, and

TABLE VI
CUTTING EACH ROUTE BETWEEN THE FUTURE OBJECTIVE AND THE ACTION, AT A SINGLE SEED.

Task	MIP	Back-cut	Fwd-cut	JEPA Policy
MokaMoka	90.0	100.0	90.0	95.0
MugMug	92.5	95.0	92.5	95.0
Square	92.5	97.5	97.5	97.5
Tool Hang	65.0	17.5	55.0	65.0
Transport (4cam)	97.5	95.0	97.5	97.5
Coffee Preparation	30.0	65.0	70.0	70.0
Kitchen	62.5	62.5	72.5	77.5
Three Piece Assembly	60.0	52.5	52.5	65.0
Mean (8 tasks)	73.8	73.1	78.4	82.8

All four arms use seed 42 and the same code base, so the comparison is seed-matched. These numbers are *not* comparable with the three-seed means in Table II. *Back-cut* preserves attention from action tokens to the future token but detaches the future loss from the shared trunk. *Fwd-cut* allows the future loss to shape the shared representation but removes the future token from the action tokens’ attention. The Transport cells use the aligned 10k-230k window; all maxima occur before its boundary.

TABLE VII
FUTURE-LOSS RATIO (FIXED $k = 4$, I.E., 0.2 s AT 20 HZ) AND HORIZON (FIXED $\rho = 0.10$) ABLATIONS (BEST-CHECKPOINT SUCCESS RATE %, THREE-SEED MEAN $\pm$ STD).

Config.	MokaMoka	MugMug	Square	Tool Hang	Transport (4cam)
baseline	86.7 $\pm$ 2.9	94.2 $\pm$ 2.9	94.2 $\pm$ 1.4	65.8 $\pm$ 6.3	97.5 $\pm$ 0.0
$\rho = 0.05$	93.3 $\pm$ 5.2	92.5 $\pm$ 2.5	95.0 $\pm$ 0.0	70.8 $\pm$ 8.0	99.2 $\pm$ 1.4
$\rho = 0.10$	92.5 $\pm$ 2.5	96.7 $\pm$ 1.4	95.8 $\pm$ 1.4	75.0 $\pm$ 9.0	98.3 $\pm$ 1.4
$\rho = 0.20$	91.7 $\pm$ 1.4	97.5 $\pm$ 2.5	97.5 $\pm$ 0.0	70.8 $\pm$ 8.0	99.2 $\pm$ 1.4
$k = 2$	90.0 $\pm$ 0.0	94.2 $\pm$ 2.9	95.0 $\pm$ 0.0	69.2 $\pm$ 13.8	99.2 $\pm$ 1.4
$k = 6$	93.3 $\pm$ 2.9	95.0 $\pm$ 2.5	98.3 $\pm$ 2.9	70.8 $\pm$ 6.3	100.0 $\pm$ 0.0

Table II shows substantial per-task variation across seeds. Back-cut matches the aggregate baseline through large offsetting changes: -47.5 points on Tool Hang and $+35.0$ on Coffee Preparation. The seed-42 Coffee baseline is the collapsed run. We therefore claim only that severing the backward route removes the aggregate gain and destabilizes per-task behavior. We attach no significance statement to a single seed.

F. Future-Loss Ratio and Horizon

Table VII and Fig. 9 sweep the two hyperparameters of the future branch. No configuration with future supervision is materially worse than the baseline: across the five tasks and six settings, the only cell below the baseline is $\rho = 0.05$ on MugMug (92.5 vs. 94.2), a 1.7-point difference smaller than one evaluation episode. The method is otherwise insensitive within the swept range. $\rho = 0.10$ – 0.20 dominates $\rho = 0.05$ on the precision-heavy tasks (MugMug, Square, Tool Hang), and $k = 4$ – 6 dominates $k = 2$, consistent with the selection principle of Sec. III-A: too short a horizon makes the future target nearly redundant with the current observation. Tool Hang is the most sensitive task (best at $\rho = 0.10$, $k = 4$), while Square and Transport favor the larger $k = 6$.

G. Robustness to the Observation Configuration

Table VIII varies the camera configuration at seed 42. With wrist images only, six of seven tasks improve and one ties; MokaMoka and Coffee Preparation each gain 10.0 points. With

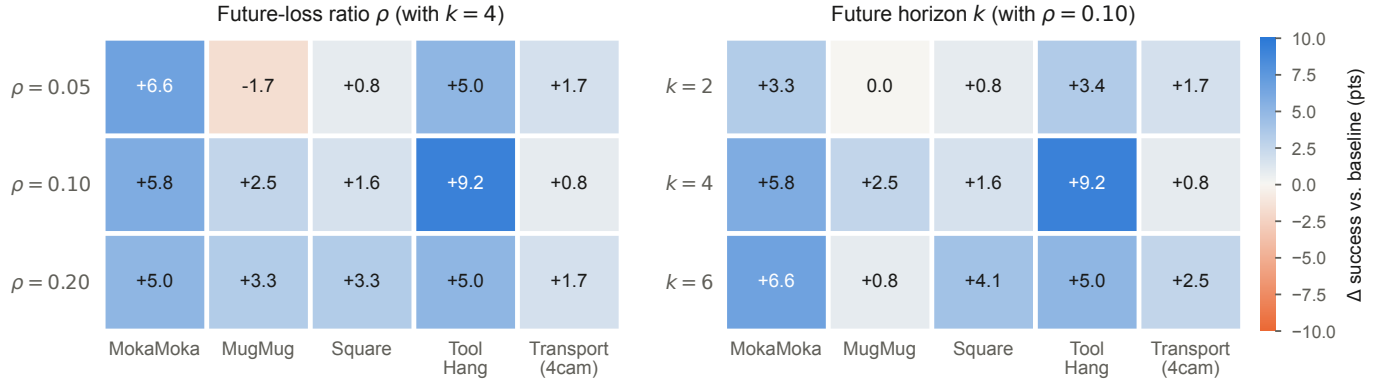


Fig. 9. Hyperparameter sensitivity as gain over the baseline (three-seed means, in points): future-loss ratio ρ (left) and future horizon k (right); blue is better than the baseline, orange worse. No configuration is materially worse than the baseline; $\rho \in [0.1, 0.2]$ and $k \in \{4, 6\}$ are uniformly strong choices.

TABLE VIII
CAMERA-CONFIGURATION SWEEP (BEST-CHECKPOINT SUCCESS RATE %, SINGLE SEED 42; DIRECTIONAL ONLY).

Config.	Moka	Mug	Square	Tool Hang	Coffee	Kitchen	Three Piece
Standard, baseline	90.0	92.5	92.5	65.0	30.0	62.5	60.0
Standard, future-4	95.0	95.0	97.5	65.0	70.0	77.5	65.0
Wrist only, baseline	62.5	80.0	80.0	27.5	37.5	42.5	15.0
Wrist only, future-4	72.5	87.5	80.0	30.0	47.5	47.5	17.5
Fixed only, baseline	87.5	70.0	90.0	35.0	67.5	65.0	42.5
Fixed only, future-4	N/A	N/A	87.5	37.5	62.5	62.5	40.0

Transport is omitted because the camera sources differ across rows. The fixed-only future-4 LIBERO cells are marked N/A because a data bug invalidated that batch.

fixed cameras only, all five usable differences remain within ± 5 points, so the gain is not reproduced. These results are directional: an identical single-seed configuration varied by 17.5 points between launch batches. The fixed-camera results therefore provide insufficient evidence of a reversal. The main comparison instead averages three seeds (Sec. IV-C).

H. Alternative Designs and Negative Results

Explicit isotropic regularization is fragile. Adding the SIGReg isotropic-Gaussian objective [50] (weight 0.09) lowers performance on all five tasks (Table IX), with MokaMoka at 0.0 across all seeds despite normal training completion. The prior may over-regularize features already constrained by action supervision; without a weight sweep, this mechanism remains a hypothesis and does not imply that SIGReg generally fails.

State inputs. With low-dimensional state inputs instead of images (three seeds), the effect is task-dependent: Transport improves markedly ($52.5 \rightarrow 62.5$) while Tool Hang dips within one granularity step ($75.0 \rightarrow 72.5$). Future supervision thus systematically harms neither observation modality, and its benefit is not exclusive to image inputs.

I. Representation Collapse Audit

We evaluated collapse with a pre-specified spectral protocol. A model can avoid a constant output yet still lose rank as part of its embedding spectrum approaches zero [21]. A test that only checks whether outputs differ would miss this failure

mode. The audit covers JEPA Policy and matched action-only controls on seven tasks and three seeds, for 42 runs in total. We evaluate the best and latest checkpoint of each run, giving 84 checkpoints and 168 representation matrices over fixed 4,096-sample manifests. All metrics are computed in float64. They include effective rank, top-eigenvalue share, and tail energy TE32 with a pre-specified low-rank boundary of 0.05. A fixed 256-candidate retrieval test measures whether predictor outputs depend on the input. TE32 is the share of centered spectral energy outside the leading 32 directions. Let $X_c \in \mathbb{R}^{N \times D}$ be the mean-centered representation matrix and $\lambda_1 \geq \dots \geq \lambda_D$ the eigenvalues of $X_c^\top X_c$. Then $\text{TE32} = 1 - \sum_{i \leq 32} \lambda_i / \sum_i \lambda_i$. Small values indicate concentration in few directions. The pre-specified boundary $\text{TE32} = 0.05$ marks the point at which 32 directions contain 95% of the centered energy.

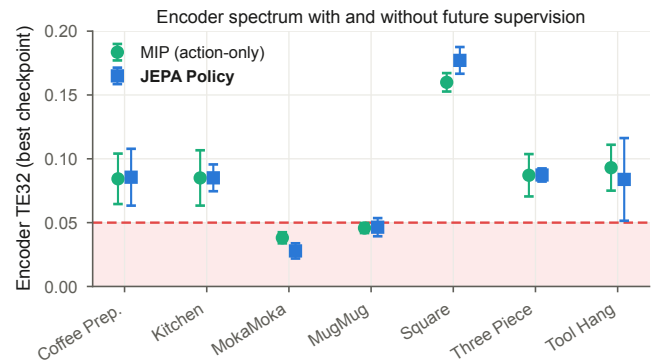


Fig. 10. Encoder TE32 at the best checkpoint (marker: three-seed mean; bar: worst seed; shading: below the pre-specified 0.05 boundary). JEPA Policy tracks MIP, including the MokaMoka and MugMug boundary exceptions.

The outcome supports a bounded claim: there is *no evidence of complete collapse or of systematic contraction to the pre-*

TABLE IX
ADDING AN EXPLICIT SIGREG REGULARIZER (WEIGHT 0.09) TO JEPA POLICY (BEST-CHECKPOINT SUCCESS RATE %, THREE-SEED MEAN $\pm$ STD).

Config.	MokaMoka	Square	Tool Hang	Coffee	Three Piece
JEPA Policy	92.5 $\pm$ 2.5	95.8 $\pm$ 1.4	75.0 $\pm$ 9.0	77.5 $\pm$ 7.5	64.2 $\pm$ 3.8
+ SIGReg ($w=0.09$)	0.0 $\pm$ 0.0	42.5 $\pm$ 9.0	2.5 $\pm$ 0.0	12.5 $\pm$ 11.5	36.7 $\pm$ 2.9

TABLE X
ROLLOUT AUDIT: EARLY FUTURE-4 NMSE VS. FAILURE (24 CHECKPOINTS; 5,200 EPISODES). EPISODE-RESAMPLED CONFIDENCE INTERVALS ARE DESCRIPTIVE CONDITIONAL ON THESE CHECKPOINTS.

Task	Episodes	Failure rate	AUROC [95% CI]	PR-AUC
MugMug	600	14.5%	0.754 [0.693, 0.810]	0.377
Tool Hang	600	30.8%	0.722 [0.676, 0.766]	0.532
Coffee Preparation	600	38.8%	0.662 [0.617, 0.706]	0.566
Three Piece Assembly	600	47.7%	0.624 [0.578, 0.667]	0.588
Transport	1,000	7.3%	0.623 [0.566, 0.681]	0.127
Square	600	10.8%	0.620 [0.544, 0.694]	0.167
Kitchen	600	21.8%	0.572 [0.513, 0.629]	0.322
MokaMoka	600	16.7%	0.554 [0.488, 0.618]	0.254

specified low-rank boundary, and predictor retrieval is far above chance (input-dependent predictions); Fig. 10 shows representative spectra. The pre-specified task-level exception rule did trigger. Encoder TE32 for MugMug and MokaMoka, and predictor TE32 for Tool Hang and Square, crossed or touched the boundary at specific endpoints. We therefore do not claim that every task avoids low-rank contraction. A non-collapsed point estimate alone would not establish that the action loss supplies the constraint, because the same optimization might remain stable without it. For this reason, the audit and the $\lambda_a = 0$ control answer complementary questions. The multi-task audit tests whether collapse occurs under the complete training objective, while the removal control tests what changes when action supervision is absent. A complementary trajectory experiment used four 50k-step runs with snapshots at $\{0, 1k, 5k, 10k, 25k, 50k\}$ steps. During the first 1,000 steps, the fixed future loss fell by 97.7% and all seeds crossed the low-rank boundary. TE32 then partially recovered instead of decreasing monotonically. Retrieval Recall@5 remained far above chance at every endpoint (permutation $p = 1/1001$).

Removing the action loss collapses the representation.

The audit above shows only that collapse does not occur when the action loss is present. We scored the $\lambda_a = 0$ variant from Sec. IV-B under the same protocol. This variant receives no action-loss gradient, while the future loss still trains the encoder and shared trunk. Encoder TE32 falls from 0.170 to 0.0014, about $36\times$ below the pre-specified 0.05 boundary. Centered effective rank decreases from 181 to 24 of 384. In addition, 92% of the embedding energy moves into the shared mean. The cosine similarity between two samples rises to 0.918, compared with 0.021 in the matched normal run. The collapse is not complete because predictor retrieval remains far above chance (Recall@1 0.976, permutation $p = 0.001$). This value is higher than the 0.525 of the normal run, consistent with a future representation that becomes nearly deterministic within a 24-dimensional subspace. The future loss alone therefore admits a degenerate solution in this experiment, whereas the action loss prevents it. This control covers only one task and one seed.

J. Future-Consistency Diagnostic at Rollout

We evaluated the diagnostic from Sec. III-F on 24 frozen checkpoints from eight tasks and three seeds. Checkpoint selection used no audit metric. The evaluation contains 5,200

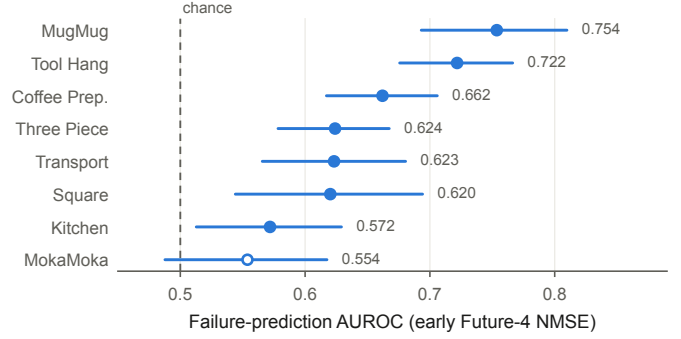


Fig. 11. Per-task failure-prediction AUROC of the early Future-4 NMSE with conditional episode-level 95% confidence intervals. The open marker (MokaMoka) denotes a confidence interval that crosses chance.

rollout episodes, including 4,040 successes and 1,160 failures. Before analysis, we verified byte-identical episode evidence across worker counts after correcting a reset-seed bug. Each episode is scored by the median Future-4 NMSE (a 0.2 s-ahead prediction; Sec. IV-A) of its first five policy decisions, i.e., by whether the future prediction is already inconsistent *early* in the episode. For one decision, the NMSE is $d_{\text{NMSE}}(\hat{\mathbf{z}}_{t+k}^{(1)}, \mathbf{z}_{t+k})$ as defined in Sec. III-C. The normalization makes episodes comparable within a task despite different latent scales. An episode enters the analysis only when its first five decisions all have valid future targets. The 95% intervals use 10,000 episode-level percentile-bootstrap resamples with a fixed seed. They are descriptive conditional on the three evaluated checkpoints per task, not checkpoint-population inference. Episode-level two-sided permutation tests use 10,000 resamples; checkpoint robustness is reported separately below.

The error ranks failures. Pooled within each task, failures have higher early NMSE than successes on all eight tasks. AUROC exceeds 0.5 throughout, and seven conditional episode-level intervals exclude chance (Table X, Fig. 11). MugMug (0.754) and Tool Hang (0.722) have the highest AUROC values. Only the MokaMoka interval crosses chance. On MugMug, failures outrank successes in 75.4% of random failure-success pairs. At the checkpoint level, 22 of 24 point estimates exceed 0.5 and 23 of 24 leave-one-seed-out tests agree in direction. The pooled Transport value is not stable at the checkpoint level because two of its three seeds are individually below 0.5.

We use threshold-independent AUROC, following established protocols for misclassification and out-of-distribution detection [39]. The computational cost differs from ensemble uncertainty. Our score comes from a policy forward pass that is already required, whereas an ensemble needs M trained networks and M passes per decision [40]. AUROC measures whether a randomly selected failure tends to receive a larger score than a randomly selected success. It does not choose an alarm threshold and does not imply that the same numerical score has the same meaning across tasks. This distinction is central here because the predictor is useful for within-task ranking even when absolute error scales differ.

Two controls rule out trivial explanations. Residualizing the NMSE against the magnitude of realized latent visual change leaves the signal unchanged (checkpoint-macro AUROC

0.6184 $\rightarrow$ 0.6183). The association is therefore not explained by failures producing more motion. A persistence baseline that sets the future latent equal to the current latent is near chance (0.508 vs. 0.618 for the learned predictor). The signal therefore depends on the learned prediction rather than generic latent motion alone.

Raw scores must be conditioned per task. Pooling raw NMSE across tasks removes the signal (AUROC 0.501, CI [0.484, 0.519]; permutation $p = 0.68$). This is a Simpson-type effect caused by task-specific error scales. Any monitoring application must therefore calibrate per task (and for some tasks per checkpoint). Post-hoc Youden thresholds are descriptive because they are chosen and evaluated on the same data. MugMug transfers best across seeds (held-out balanced accuracy 0.69–0.76), and Coffee transfers moderately. Kitchen and MokaMoka support no useful raw threshold. Tool Hang and Transport show usable ranking but a drifting absolute scale. We use the diagnostic as a per-task-calibrated risk signal for failure analysis and human hand-off, not as a failure probability. It is learned from successful demonstrations and requires no foundation model, separate scoring network, or extra action samples. Dedicated runtime monitors use a different design [42], [43]. Operational thresholds still require held-out rollouts labeled by outcome. Because we ran no direct comparison, our claims concern cost and signal type rather than detection accuracy. Thus, the positive AUROC result establishes an ordering signal, whereas deployment requires a second calibration stage on outcome-labeled data from the intended task and hardware. Keeping these two stages separate avoids turning a retrospective diagnostic into an unsupported claim of calibrated online failure prediction.

K. Real-Robot Evaluation

We repeated the comparison on a physical two-arm ARX-5; one NVIDIA GeForce RTX 3090 ran inference from three 128×128 RGB streams, with the driver at 30 Hz and actions at 10 Hz. Across 3,048 steady-state JEPA Policy chunks from 160 episodes (first chunk excluded), frame-to-chunk latency was 20.1/63.3 ms (median/ p_{95}). MIP/DP-16/DP-100 recorded 27.6/67.5, 79.2/130.8 and 316.2/373.3 ms (median/ p_{95}). All methods used a synchronous rollout loop: each policy call completed before its returned action chunk was executed, and action execution did not overlap the next policy call. JEPA Policy’s 20.1 ms median latency was well below the 100 ms action period, so it required no asynchronous action queue. JEPA Policy and MIP each executed eight actions per chunk, whereas DP-16/DP-100 skipped one/three expired positions, leaving eight/six executable actions. The five tasks are Cabinet, Cup Stack, Cup Upright, Plate Grape, and Pen Insert (Fig. 12); their arm assignments are reported with Table XI. Success required completing the operator-confirmed task goal within a fixed task-specific action-chunk budget shared by all methods; wall-clock duration, inference latency, and control-loop timing did not affect the label. Each policy was evaluated at 60k, 100k, and 140k steps for 10 episodes per checkpoint. Diffusion Policy used 16 and 100 denoising steps. All methods covered all tasks; the 63 sessions total 630 episodes without exclusions.

Reproducibility and release. The anonymous repository linked on the first page already includes the complete real-robot deployment stack, task launchers, safety checks, and evaluation manifests. We will release these materials publicly with the training code upon publication.

TABLE XI
REAL-ROBOT SUCCESS ON FIVE TASKS (THREE CHECKPOINTS PER ARM;
NOMINALLY 10 EPISODES PER CELL; THREE ACCIDENTAL REPEATS
POOLED).

Task	Steps	DP-100	DP-16	MIP	JEPA Policy
Cabinet	60k	0/10	0/10	2/10	5/10
	100k	0/10	0/10	3/10	6/10
	140k	0/10	0/10	4/10	7/10
	all	0%	0%	30%	60%
Cup Stack	60k	1/10	8/20	8/10	6/10
	100k	1/10	5/20	2/10	7/10
	140k	1/10	3/10	2/10	4/10
	all	10%	32%	40%	57%
Cup Upright	60k	1/10	2/10	4/10	5/10
	100k	5/10	3/10	8/10	14/20
	140k	2/10	7/10	6/10	10/10
	all	27%	40%	60%	72%
Plate Grape	60k	0/10	0/10	5/10	2/10
	100k	1/10	0/10	5/10	5/10
	140k	2/10	3/10	5/10	7/10
	all	10%	10%	50%	47%
Pen Insert	60k	4/10	6/10	8/10	10/10
	100k	3/10	8/10	10/10	10/10
	140k	7/10	9/10	10/10	9/10
	all	47%	77%	93%	97%
Pooled (episode-weighted)		18.7%	31.8%	54.7%	66.9%

DP-100 and DP-16 are Diffusion Policy at 100 and 16 denoising steps. The same two-arm platform runs every task; Cup Stack, Cup Upright and Pen Insert are performed with the right arm alone, Cabinet and Plate Grape with both. Three conditions were repeated by accident and both sessions are pooled, which is why those cells report 20 episodes: Cup Stack DP-16 at 60k and 100k, and Cup Upright JEPA Policy at 100k.

The pooled ordering matches the simulation results. Table XI reports every cell. Episode-weighted success is 66.9% for JEPA Policy, 54.7% for MIP, 31.8% for DP-16, and 18.7% for DP-100. Across matched task-checkpoint cells, JEPA Policy leads DP-16 in 14 of 15 cells and DP-100 in all 15. It is never behind either configuration. The mean margins are +35 and +48 points, with two-sided sign-test values of $p = 1.22 \times 10^{-4}$ and 6.10×10^{-5} . Because checkpoints within a task are correlated, these tests describe consistency rather than independent-task inference. Against MIP, JEPA Policy leads in 9 of 15 cells, trails in 4, and ties in 2. The mean margin is +12.0 points and is not significant (sign test $p = 0.27$). The cell counts should therefore not be read as 15 independent replications of each method. They show whether the ordering persists across the three reported checkpoints, while the five tasks remain the broader units of variation. The margin over Diffusion Policy is larger than the +7.9 points observed in simulation, where one task accounts for much of the difference. The margin over MIP has the same direction as the simulated +5.6-point gain, but its task distribution differs. It is concentrated in Cabinet (+30.0), while Plate Grape favors MIP (−3.3) and Pen Insert is nearly tied (+3.3, with both above 90%).

Cabinet separates the arms by failure stage. Neither

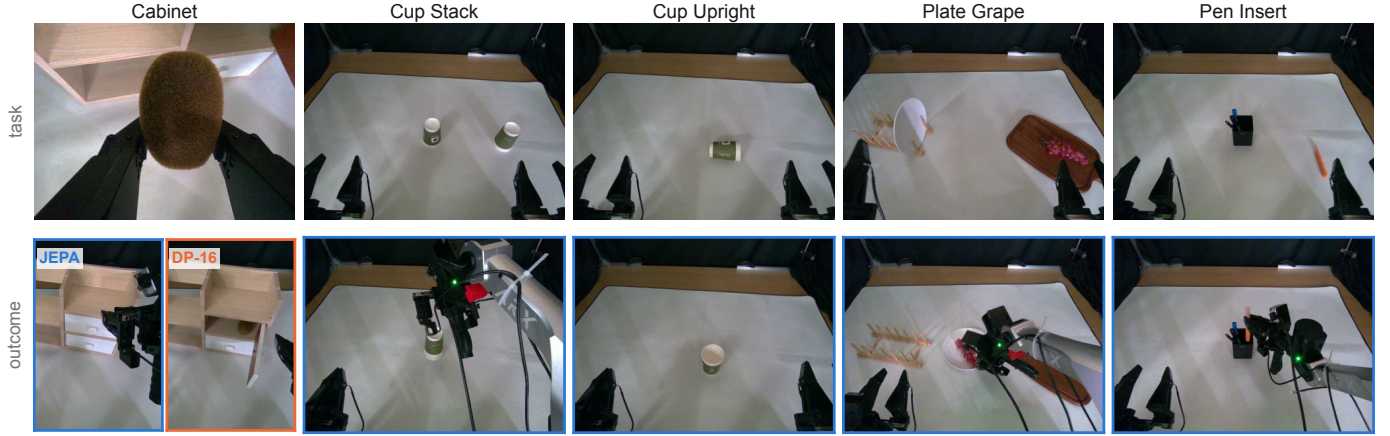


Fig. 12. Representative task and outcome frames from successful JEPA Policy episodes on the five real-robot tasks. The Cabinet task frame uses the left-wrist camera to show the kiwi grasp; the remaining frames use the base camera. The Cabinet outcome cell additionally contrasts the JEPA Policy success (door closed) with a DP-16 terminal failure (door left open). Policies receive all views resized to 128×128 .

Diffusion Policy configuration completes a Cabinet episode, but the failures occur at different stages. We define the final stage as at least 194 issued actions, the lower quartile among successful episodes. Under this definition, 7 of 30 DP-16 failures and 2 of 30 DP-100 failures placed the object but did not close the door. Video inspection confirms these outcomes (Fig. 12); the remaining failures occurred during approach or grasping. Door closure is required for every arm. Seven of 21 MIP failures also stopped at this stage, and one successful MIP episode required three closing attempts. None of the 12 JEPA Policy failures occurred at the closing stage. Whenever JEPA Policy reached the door, it closed it. Plate Grape shows a similar pattern. Using a task-specific threshold of 254 actions, 3 of 15 MIP failures reached the final stage, compared with 0 of 16 for JEPA Policy and 0 of 27 for DP-16. DP-16 never reached the transfer stage on this task.

V. DISCUSSION

The experiments indicate that future supervision within the action-generating stack improves manipulation success with little added latency. JEPA Policy adds 0.29 ms (2.2%) to the two-pass MIP baseline. Its 13.2 ms decision time is $33\times$ lower than that of Diffusion Policy in the evaluated configuration (Table III). The shared-versus-separate ablation provides the most direct evidence for the proposed topology. Applying the same loss through a separate branch, with or without cross-attention, provides little or no benefit. The gain therefore depends on how the future objective is integrated rather than on the mere presence of an auxiliary loss. The Dual-independent arm reproduces the property of ACT-JEPA [6] on which this test turns: a shared conditioning path trained by both losses, feeding two mutually invisible branches. Holding target modality, anti-collapse mechanism and training recipe fixed leaves topology as the only changed variable.

The scope of these claims is limited by the observed variation across tasks. JEPA Policy leads Diffusion Policy on all nine tasks, but Tool Hang accounts for much of the mean difference. Without Tool Hang, the mean difference is +5.4 points. We therefore interpret the accuracy result as comparable or better

performance under the evaluated settings. The clearer advantage is computational: the measured latency differs by a factor of 33. The collapse audit supports a narrower statement. We observe no complete collapse or systematic contraction to the pre-specified boundary, although some tasks show low-rank exceptions. In the one-task $\lambda_a = 0$ control, removing the action loss produces a boundary violation of about $36\times$. This result provides direct evidence that the action loss constrains the representation in that setting. The SIGReg [50] experiment further shows that the tested explicit regularizer can severely reduce policy performance. The rollout audit shows the future branch carries real action-outcome information, but only as a *per-task-calibrated* risk signal. Raw scores do not transfer across tasks, and post-hoc thresholds remain descriptive.

Limitations. Most experiments are conducted in simulation, and the hardware study is smaller. It covers five tasks, three checkpoints, ten episodes per cell, one robot, and one operator in one laboratory. Individual cells are noisy at this sample size. One condition that was repeated by accident returned 5/10 and 9/10 for the same checkpoint and scene. The comparison with Diffusion Policy is consistent across cells (29 of 30 ahead and none behind), whereas the advantage over MIP is only directional. Generalization beyond the three evaluated benchmark suites remains untested. Table III isolates model time on one PPU, task, and seed with different PyTorch runtimes; RTX 3090 logs add shared-system end-to-end latency including sensing and middleware. Like-for-like comparison against few-step action generators, distilled [13] or single-stage [14], remains open. The best-checkpoint protocol, standard in this literature, is optimistic by construction, although Table IV shows the reported gain survives three selection-free protocols. Single-seed side studies are directional only, and multi-modal futures remain untested.

VI. CONCLUSION

JEPA Policy couples expert actions with their demonstrated future representations in a shared Transformer and predicts both in two diffusion-free passes. Across nine simulated tasks, it improved the action-only MIP mean by 5.6 points and exceeded

both MIP and the evaluated Diffusion Policy configurations on every task. It also reduced measured decision latency from 439.5 to 13.2 ms. Ablations attributed the gain to the shared stack, and the 84-checkpoint audit found no complete collapse with action supervision. The 5,200-episode rollout audit supported a per-task-calibrated rather than universal risk signal, while the five-task, 630-episode hardware study preserved the pooled method ordering. Future work should broaden real-robot coverage, compare matched few-step generators [13], [14], validate diagnostic thresholds on held-out rollouts, and quantify multi-modal futures.

REFERENCES

- [1] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems*, 1988.
- [2] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [3] C. Chi *et al.*, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Robotics: Science and Systems (RSS)*, 2023.
- [4] C. Pan *et al.*, "Much ado about noising: Dispelling the myths of generative robotic control," in *International Conference on Learning Representations (ICLR)*, 2026.
- [5] M. Assran *et al.*, "Self-supervised learning from images with a joint-embedding predictive architecture," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [6] A. Vujinovic and A. Kovacevic, "ACT-JEPA: Novel joint-embedding predictive architecture for efficient policy representation learning," *IEEE Access*, vol. 14, pp. 78 895–78 906, 2026.
- [7] T. Z. Zhao *et al.*, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Robotics: Science and Systems (RSS)*, 2023.
- [8] A. Mandlekar *et al.*, "What matters in learning from offline human demonstrations for robot manipulation," in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 164, 2021, pp. 1678–1690.
- [9] P. Florence *et al.*, "Implicit behavioral cloning," in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 164, 2021, pp. 158–168.
- [10] N. M. M. Shafiullah *et al.*, "Behavior transformers: Cloning k modes with one stone," in *Advances in Neural Information Processing Systems*, 2022.
- [11] Y. Song *et al.*, "Consistency models," in *International Conference on Machine Learning (ICML)*, 2023.
- [12] Y. Lipman *et al.*, "Flow matching for generative modeling," in *International Conference on Learning Representations (ICLR)*, 2023.
- [13] A. Prasad *et al.*, "Consistency policy: Accelerated visuomotor policies via consistency distillation," in *Robotics: Science and Systems (RSS)*, 2024.
- [14] K. Frans *et al.*, "One step diffusion via shortcut models," in *International Conference on Learning Representations (ICLR)*, 2025.
- [15] Y. Ze *et al.*, "3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations," in *Robotics: Science and Systems (RSS)*, 2024.
- [16] Y. LeCun, "A path towards autonomous machine intelligence, version 0.9.2," OpenReview, 2022, position paper.
- [17] J.-B. Grill *et al.*, "Bootstrap your own latent - a new approach to self-supervised learning," in *Advances in Neural Information Processing Systems*, 2020.
- [18] A. Bardes, J. Ponce, and Y. LeCun, "VICReg: Variance-invariance-covariance regularization for self-supervised learning," in *International Conference on Learning Representations (ICLR)*, 2022.
- [19] J. Zbontar *et al.*, "Barlow twins: Self-supervised learning via redundancy reduction," in *International Conference on Machine Learning (ICML)*, 2021.
- [20] X. Chen and K. He, "Exploring simple Siamese representation learning," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 15 750–15 758.
- [21] L. Jing *et al.*, "Understanding dimensional collapse in contrastive self-supervised learning," in *International Conference on Learning Representations (ICLR)*, 2022.
- [22] Y. Tian, X. Chen, and S. Ganguli, "Understanding self-supervised learning dynamics without contrastive pairs," in *International Conference on Machine Learning (ICML)*, 2021.
- [23] H. Wu *et al.*, "Unleashing large-scale video generative pre-training for visual robot manipulation," in *International Conference on Learning Representations (ICLR)*, 2024.
- [24] Y. Tian *et al.*, "Predictive inverse dynamics models are scalable learners for robotic manipulation," in *International Conference on Learning Representations (ICLR)*, 2025.
- [25] A. Brohan *et al.*, "RT-1: Robotics transformer for real-world control at scale," in *Robotics: Science and Systems (RSS)*, 2023.
- [26] B. Zitkovich *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 2165–2183.
- [27] M. J. Kim *et al.*, "OpenVLA: An open-source vision-language-action model," in *Conference on Robot Learning (CoRL)*, 2024.
- [28] Octo Model Team *et al.*, "Octo: An open-source generalist robot policy," in *Robotics: Science and Systems (RSS)*, 2024.
- [29] Open X-Embodiment Collaboration, "Open X-Embodiment: Robotic learning datasets and RT-X models," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6892–6903.
- [30] K. Black *et al.*, " π_0 : A vision-language-action flow model for general robot control," in *Robotics: Science and Systems (RSS)*, 2025.
- [31] J. Sun *et al.*, "VLA-JEPA: Enhancing vision-language-action model with latent world model," in *European Conference on Computer Vision (ECCV)*, 2026.
- [32] S. Miao *et al.*, "JEPA-VLA: Video predictive embedding is needed for VLA models," *arXiv preprint arXiv:2602.11832*, 2026.
- [33] D. Ha and J. Schmidhuber, "World models," *arXiv preprint arXiv:1803.10122*, 2018.
- [34] M. Assran *et al.*, "V-JEPA 2: Self-supervised video models enable understanding, prediction and planning," *arXiv preprint arXiv:2506.09985*, 2025.
- [35] M. Laskin, A. Srinivas, and P. Abbeel, "CURL: Contrastive unsupervised representations for reinforcement learning," in *International Conference on Machine Learning (ICML)*, 2020.
- [36] M. Schwarzer *et al.*, "Data-efficient reinforcement learning with self-predictive representations," in *International Conference on Learning Representations (ICLR)*, 2021.
- [37] D. Hafner *et al.*, "Mastering diverse control tasks through world models," *Nature*, vol. 640, no. 8059, pp. 647–653, 2025.
- [38] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *International Conference on Learning Representations (ICLR)*, 2024.
- [39] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *International Conference on Learning Representations (ICLR)*, 2017.
- [40] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems*, 2017.
- [41] L. Lindemann *et al.*, "Safe planning in dynamic environments using conformal prediction," *IEEE Robotics and Automation Letters*, vol. 8, no. 8, pp. 5116–5123, 2023.
- [42] C. Agia *et al.*, "Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress," in *Conference on Robot Learning (CoRL)*, 2024.
- [43] C. Xu *et al.*, "Can we detect failures without failure data? Uncertainty-aware runtime failure detection for imitation learning policies," in *Robotics: Science and Systems (RSS)*, 2025.
- [44] B. Liu *et al.*, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.
- [45] A. Mandlekar *et al.*, "MimicGen: A data generation system for scalable robot learning using human demonstrations," in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 1820–1864.
- [46] Y. Zhu *et al.*, "robosuite: A modular simulation framework and benchmark for robot learning," *arXiv preprint arXiv:2009.12293*, 2020.
- [47] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [48] K. He *et al.*, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [49] M. Oquab *et al.*, "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024.
- [50] R. Balestriero and Y. LeCun, "LeJEPA: Provable and scalable self-supervised learning without the heuristics," *arXiv preprint arXiv:2511.08544*, 2025.